NAZARBAYEV UNIVERSITY

Programming for Engineers

Detailed Reading Notes

Week 7: Testing and Debugging Exceptions and Assertions

Yanwei Wang

School of Engineering

Nazarbayev University

Fall 2026

Specify $\longrightarrow$ **Test** $\longrightarrow$ **Diagnose**
Repair $\longrightarrow$ **Retest**

SCHOOL OF ENGINEERING

About these notes

These notes expand Yanwei Wang's supplied 87-page presentation *Programming for Engineers: Week 7 — Testing, Debugging, Exceptions, and Assertions*. The slide deck defines the scope, terminology, and sequence. Read the explanations and commented examples alongside a Python interpreter, and attempt the activities before their worked answers.

Course information

Course: ENG 101 — Programming for Engineers, lecture section 2L, 6 ECTS credits.
Instructor: Dr. Yanwei Wang, School of Engineering, Nazarbayev University.
Term: Fall 2026.
Week 7 sequence: Lecture 19 — Testing and Evidence of Correctness; Lecture 20 — Systematic Debugging and Error Messages; Lecture 21 — Exceptions, Assertions, and Reliable Programs.

Finding the corresponding slide. A grey alignment line identifies the *source PDF pages*, counting the cover and lecture dividers. Those numbers differ from some slide-footer numbers. Appendix B maps all 87 source pages to sections of this document.

Code and output. Gold listings contain Python; green comments explain intent, units, and control flow. Do not type the line numbers. Grey listings contain outputs or sample interactions. Each example has a separate `.py` file; any repeated prerequisite definition is identified in comments.

Source and solutions. Slide examples retain their behavior, with comments added and PDF line wraps repaired. New repairs, solutions, and completed implementations are explicitly labelled. Faulty examples appear before their repairs; a wrong answer without an exception remains wrong.

Two source limitations are made explicit

Source page 49 names `debug_lab_starter.py` without displaying its contents. These notes provide a *reconstructed activity* matching the three described defect types, not a recovered copy of that file. Pages 74–76 give a contract and an excerpt of an integrated stress report, but omit its initial type and finiteness checks. The complete implementation in Section 3.19 is a labelled completion of that contract.

Failure demonstrations. Run intentional-error files individually. Other explicitly labelled examples run but give incorrect results. The combined demonstration excludes both kinds of defect and the interactive loops, so it finishes without keyboard input.

Assessment note

These are reading and practice materials, not new assessed tasks. The Week 7 slides do not change examination coverage, assignment weights, deadlines, or permitted-resource rules. Follow the separate Moodle announcements, as stated on source page 84.

How to study Week 7

In earlier weeks you learned to make a program compute something. This week asks a harder question: *what evidence supports that computation?* A result may have the expected type and a plausible magnitude while still being wrong because of a missed boundary, a unit conversion, an unintended data change, or an unsuitable physical assumption.

Lecture	Main question	What to record
19	What should happen, and which tests challenge that promise?	Inputs, independent expectations, actual results, and acceptance criteria.
20	Where does execution first disagree with the expected state?	Reproduction steps, tracebacks, intermediate values, hypotheses, and repairs.
21	How should a program respond when no justified result exists?	Input contracts, expected exceptions, successful paths, and repeatable assertions.

A practical reading cycle

Before each example, state what the inputs mean and predict the output or exception. Trace only the values relevant to the question. Then run the file from a clean session and compare the observation with your prediction. When a defect is found, explain the cause before changing code. Make the smallest justified repair and keep the failing input as a regression test.

For a working function, try an ordinary case, an exact boundary, and an uncomfortable case such as an empty list, a zero denominator, or a missing key. Do not assume that a function validates something merely because its comments say the caller should supply it.

Key idea

Specify → Test → Diagnose → Repair → Retest. Testing is not a final cosmetic stage. It is a way to make an engineering claim inspectable throughout program development.

Prerequisites and reading depth

You will reuse conditions and loops from Week 2, unit-aware numerical reasoning from Weeks 1–3, functions from Week 4, list copying from Week 5, and recursion and dictionaries from Week 6. No object-oriented programming, custom exception classes, or external testing framework is required. The slide extensions on `finally`, relative tolerance, and graphical debuggers remain optional and are collected in Chapter 4.

Recommended order. Read Chapters 1–3 in lecture order. Attempt the activities before their worked answers. Use Chapter 5 for a later self-check, and the glossary and source-page map when revising. A test suite is a set of selected experiments, not a certificate of universal correctness.

Contents

About these notes 2

How to study Week 7 3

1. Testing and Evidence of Correctness 6
1.1. Two passing tests can conceal one important defect . . . 6
1.2. Correctness begins with a specification . . . 7
1.3. A test is a small engineering experiment . . . 8
1.4. Choose cases for different reasons . . . 9
1.5. Black-box testing: design from the contract . . . 9
1.6. Glass-box testing: inspect paths as well . . . 10
1.7. Test repetition: zero, one, and several passes . . . 11
1.8. Recursive return paths need tests too . . . 13
1.9. Floating-point agreement needs an explicit criterion . . . 14
1.10. Engineering trends complement known answers . . . 15
1.11. Test the data and its preservation . . . 15
1.12. Integration tests challenge the interfaces . . . 15
1.13. Build a small repeatable test driver . . . 16
1.14. Worked activity: a five-case mean test plan . . . 17
1.15. Keep a repaired defect as a regression case . . . 18
1.16. Lecture 19 exit ticket and worked answers . . . 18

2. Systematic Debugging and Error Messages 20
2.1. One short script can contain several defects . . . 20
2.2. Classify the symptom before choosing a repair . . . 21
2.3. Syntax errors: inspect the structure . . . 21
2.4. Runtime failures preserve a call trail . . . 22
2.5. Common exception names guide inspection . . . 23
2.6. Debugging is a controlled investigation . . . 24
2.7. Inspect values, types, and invisible characters . . . 24
2.8. Repair the bar calculation one cause at a time . . . 25
2.9. Trace a state variable instead of only its final value . . . 28
2.10. Trace an unexpected None to the function that returned it . . . 29
2.11. Debug side effects with a saved-state test . . . 30
2.12. Debug dictionary lookups one level at a time . . . 32
2.13. Nontermination and hidden state need evidence too . . . 33
2.14. Worked activity: diagnose an entire tiny program . . . 33
2.15. Ask for useful help without transferring responsibility . . . 35
2.16. Lecture 20 exit ticket and worked answers . . . 36

3. Exceptions, Assertions, and Reliable Programs 37
3.1. What should a zero-area request produce? . . . 37
3.2. Raise an error before invalid arithmetic . . . 37
3.3. Choose an exception that explains the failure . . . 38
3.4. Handle an expected conversion error . . . 39
3.5. Worked trace: which letters are printed? . . . 40
3.6. Different failures can require different responses . . . 40
3.7. Use else to isolate successful follow-up . . . 41

3.8. A caller can handle a function's rejection . . . 42
3.9. Combine a retry loop with a deliberate exit . . . 43
3.10. Never turn every failure into plausible data . . . 44
3.11. Assertions make expectations executable . . . 45
3.12. A test function is an ordinary function . . . 46
3.13. A rejection can be the successful result of a test . . . 47
3.14. A tolerance is part of the test specification . . . 48
3.15. Input validation is not an assertion . . . 49
3.16. An internal invariant checks the program's accounting . . . 49
3.17. Complete the mean-reading test suite . . . 50
3.18. Successful conversion does not prove numerical validity . . . 52
3.19. Integrated stress report: contract, calculation, and decision . . . 53
3.20. Lecture 21 exit ticket and worked answers . . . 56

4. Optional Extensions and Week 7 Synthesis **58**
4.1. Optional: finally is for cleanup . . . 58
4.2. Optional: combine absolute and relative tolerance . . . 59
4.3. Optional: inspect state with a graphical debugger . . . 59
4.4. Worked extension: add finite-data checking to retry . . . 60
4.5. Quick reference: plan tests, then diagnose failures . . . 60
4.6. Quick reference: exceptions and assertions . . . 61
4.7. Concept map: from a promise to a reproducible report . . . 61
4.8. Scope, assessment, and preparation for the next topic . . . 62
4.9. The closing engineering principle . . . 62

5. Cumulative Practice and Worked Solutions **63**
5.1. Twelve self-check questions . . . 63
5.2. Worked solutions . . . 64
5.3. A readiness checklist . . . 66

A. Glossary and Technical Clarifications **67**
A.1. Week 7 glossary . . . 67
A.2. Clarifications and source boundaries . . . 69

B. Source Basis and Slide-by-Slide Coverage **70**
B.1. Source basis and suggested reading . . . 70
B.2. Language-reference checks . . . 70
B.3. Coverage of every source PDF page . . . 70
B.4. Using the accompanying code files . . . 72

Bibliography **74**

1 Testing and Evidence of Correctness

Lecture-slide alignment: Week 7 source PDF, pp. 3–27. PDF page numbers include title and divider pages.

Lecture 19. This lecture treats testing as the design of small, repeatable experiments. The central question from the slides is: *What evidence would convince another engineer?* A credible answer states the promise, challenges it with selected inputs, and compares observations with expectations that do not simply repeat the same code.

Learning outcomes

By the end of this chapter you should be able to specify inputs, units, results, and failure behavior; choose known-answer, typical, boundary, and invalid cases; combine black-box and glass-box approaches; distinguish unit, integration, and regression tests; and turn a prediction into a repeatable check.

1.1. Two passing tests can conceal one important defect

Lecture-slide alignment: Week 7 source PDF, pp. 4–5. PDF page numbers include title and divider pages.

The first activity gives an illustrative factor-of-safety rule: a design is classified as `"acceptable"` when FoS ≥ 2.0. The value 2.0 belongs to the acceptable category. This is a supplied classroom rule, not a universal engineering design requirement.

Slide example: faulty FoS boundary.

```
# Deliberate defect: the stated rule includes FoS = 2.0.
def classify_design(fos):
    # This strict comparison wrongly excludes the exact boundary.
    if fos > 2.0:
        return "acceptable"
    # This also receives 2.0, which is the defect to investigate.
    return "not acceptable"

# Compare these observations with independent expectations.
print(classify_design(2.5))
print(classify_design(1.5))
print(classify_design(2.0))
```

Code 1.1: Slide example: faulty FoS boundary

Example file: `ex01_fos_boundary_bug.py`

```
acceptable
not acceptable
not acceptable
```

The function definition introduces a local parameter `fos`. Its `if` condition checks a *strict* inequality. The first `return` is reached for 2.5, while the final `return` is reached for both 1.5 and 2.0. Only the last observation violates the stated rule.

Input	Expected	Observed	Conclusion
2.5	acceptable	acceptable	Passes this selected test.
1.5	not acceptable	not acceptable	Passes this selected test.
2.0	acceptable	not acceptable	Exposes the exact-boundary defect.

Smallest repair. Change `>` to `>=`; do not change the specification to agree with the existing output. Keep the boundary case for every later version.

Worked repair: include the stated boundary.

```python
# Worked repair: a finite FoS is acceptable at or above 2.0.
def classify_design(fos):
    # Equality belongs to the acceptable category in this exercise.
    if fos >= 2.0:
        return "acceptable"
    return "not acceptable"

# Retain the original examples, including the defect-revealing one.
print(classify_design(2.5))
print(classify_design(1.5))
print(classify_design(2.0))
```

Code 1.2: Worked repair: include the stated boundary

Example file: `ex02_fos_boundary_repair.py`

Engineering check

A label such as `"acceptable"` means that this particular function's stated rule is satisfied. It does not establish that the material data, loads, failure modes, or real-world model are adequate.

1.2. Correctness begins with a specification

Lecture-slide alignment: Week 7 source PDF, pp. 6–7. PDF page numbers include title and divider pages.

A **specification** describes what a program is intended to do. For a calculation, it should identify input types and physical units, valid ranges, model assumptions, returned quantities, output accuracy, and the promised behavior when input is invalid. Without that agreement, the statement "the program is wrong" has no precise reference point.

For uniform axial stress, an initial contract might say: receive a numerical force in newtons and a positive numerical area in square metres; return the signed stress in pascals as F/A. A later version may promise explicit rejection of zero area. These are different contracts: an assumption that the caller supplies valid input is not the same as a guarantee that the function checks every input.

Definition

Testing supplies selected inputs and compares actual behavior with specified expectations. **Debugging** investigates a failure, identifies its cause, repairs the defect, and checks the repaired program again.

A failed test gives a reproducible symptom; it does not automatically identify the faulty line. Conversely, a passing collection of tests gives evidence about the selected cases, not a proof covering all possible inputs. Incorrect expected answers, untested paths, and missing requirements can all survive a test suite.

The slides distinguish two questions. *Does the code implement the specified model?* is a question about implementation. *Is that model appropriate for the real situation?* asks whether the engineering assumptions are justified. Testing the division operator does not establish that a real component is loaded only axially.

1.3. A test is a small engineering experiment

Lecture-slide alignment: Week 7 source PDF, pp. 8–9. PDF page numbers include title and divider pages.

A useful test records five things: the supplied input, the expected result, the actual result, the decision criterion, and enough information to repeat the experiment. Choosing a simple input is often more effective than starting with a large realistic dataset.

For $F = 10\,\mathrm{N}$ and $A = 2\,\mathrm{m}^2$, hand calculation gives

$$\sigma = \frac{F}{A} = \frac{10}{2} = 5\,\mathrm{Pa}.$$

The numbers are intentionally easy; their purpose is to isolate the formula and its interface, not to model a particular component.

Slide example: independent expected stress.

```python
# Inputs are force in N and strictly positive area in m^2.
def calculate_stress(force_n, area_m2):
    # The uniform axial-stress model returns Pa = N/m^2.
    return force_n / area_m2

# This expectation comes from the hand calculation 10 / 2 = 5.
expected_pa = 5.0
# Invoke the implementation only after predicting the answer.
actual_pa = calculate_stress(10.0, 2.0)
# Keep the expectation, observation, and comparison visible.
print("Expected:", expected_pa, "Pa")
print("Actual:", actual_pa, "Pa")
print("Pass:", actual_pa == expected_pa)
```

Code 1.3: Slide example: independent expected stress

Example file: `ex03_known_stress.py`

```
Expected: 5.0 Pa
Actual: 5.0 Pa
Pass: True
```

The function receives the two arguments, divides them, and returns a value. `expected_pa` is a prediction made without calling the function. `actual_pa` stores what the function returned. The final comparison produces a Boolean, not a replacement result. In this selected case 5.0 is exactly representable, so direct equality is a suitable check.

Do not calculate the expected value with the same suspect function

Calling the same function twice and comparing its results can check repeatability, but it does not supply an independent known answer. Both calls can reproduce the same defect.

1.4. Choose cases for different reasons

Lecture-slide alignment: Week 7 source PDF, pp. 10–11. PDF page numbers include title and divider pages.

Test labels describe different aspects of an experiment. *Known-answer*, *typical*, *boundary*, *invalid-input*, and *sanity* tests describe why particular data are selected. *Unit*, *integration*, and *regression* tests describe the component scope or the reason a case is retained. The categories are complementary rather than mutually exclusive.

Input-selection label	Purpose	Slide example
Known-answer	Compare with an independently obtained result.	$10/2 = 5$ Pa.
Typical	Exercise ordinary intended use.	10 000 N acting over 0.0005 m^2.
Boundary	Check limits and equality.	FoS just below, exactly at, and just above 2.0.
Invalid input	Check the specified rejection behavior.	Zero or negative area.
Sanity check	Challenge sign, magnitude, or a physical trend.	Doubling area should halve stress.

A **unit test** checks one small component, such as an area-conversion function. An **integration test** checks connected components, such as conversion followed by stress calculation and reporting. A **regression test** preserves a case that previously exposed a defect, helping detect its return after later edits.

The exact FoS = 2.0 case can simultaneously be a boundary test, a unit test of a classifier, and a regression test. None of these descriptions cancels the others. Good documentation identifies what the case is intended to reveal.

1.5. Black-box testing: design from the contract

Lecture-slide alignment: Week 7 source PDF, pp. 12–14. PDF page numbers include title and divider pages.

In **black-box testing**, choose inputs from the promised behavior without inspecting the implementation. This helps reveal a requirement that the author might have forgotten to implement. You can plan these cases before writing a function body.

The temperature contract in the slides assumes a finite temperature and an ordered interval.

Both endpoints count as normal. For an interval from 20 to 30 degrees Celsius, the input space is partitioned into

$$T < 20: \text{ too low}, \qquad 20 \le T \le 30: \text{ normal}, \qquad T > 30: \text{ too high}.$$

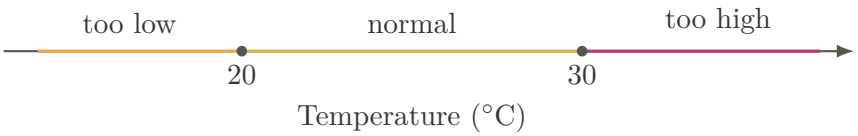


Figure 1.1.: The input partition from source page 13. The equality cases belong to the central interval.

Selecting 15, 25, and 35 degrees Celsius covers the three broad categories, but does not establish how equality is handled. The six-case matrix in the slides probes both endpoints and their nearby values.

Temperature (°C)	Expected category	What it probes
19.9	too low	Just below the lower limit.
20.0	normal	Exact inclusive lower boundary.
20.1	normal	Just inside the lower boundary.
29.9	normal	Just inside the upper boundary.
30.0	normal	Exact inclusive upper boundary.
30.1	too high	Just above the upper limit.

Here "just below" means the chosen classroom offset of 0.1 degree. It does not mean the next representable floating-point value. A test designer must choose a meaningful scale rather than assume that a particular decimal offset is universal.

1.6. Glass-box testing: inspect paths as well

Lecture-slide alignment: Week 7 source PDF, pp. 15–16. PDF page numbers include title and divider pages.

Glass-box testing examines the implementation and chooses inputs that exercise its branches, conditions, loop exits, and recursive returns. The name emphasizes that the internal structure is visible. The following is the slide classifier, with comments added.

Slide example: three temperature paths.

```python
# Contract: finite numerical inputs and low_c <= high_c.
def classify_temperature(temp_c, low_c, high_c):
    # A temperature strictly below the lower limit is too low.
    if temp_c < low_c:
        return "too low"
    # Reaching this branch already means temp_c >= low_c.
    elif temp_c <= high_c:
        return "normal"
    # The remaining valid inputs are strictly above high_c.
    return "too high"

# The exact upper boundary is included in the normal interval.
print(classify_temperature(30.0, 20.0, 30.0))
```

Code 1.4: Slide example: three temperature paths

Example file: `ex04_temperature_classifier.py`

```
normal
```

For 19.9, the first comparison is true and the first return ends the call. For 25.0, the first comparison is false but the second is true. For 30.1, both comparisons are false and the last return executes. The function does not need an explicit final `else`: reaching that return already identifies the remaining path.

Coverage is not correctness

Executing every branch shows that those branches have been exercised. It does not prove that a comparison uses the right operator or that the branch returns the right label. A classifier with `<` instead of `<=` could execute all branches and still fail at equality.

Use both viewpoints. Black-box tests ask whether the implementation matches a promise. Glass-box tests ask whether important internal routes have actually been explored. A missing requirement may not appear anywhere in the code; an unusual path may not be reached by ordinary examples.

1.7. Test repetition: zero, one, and several passes

Lecture-slide alignment: Week 7 source PDF, pp. 17–18. PDF page numbers include title and divider pages.

Loops often fail at their smallest inputs or at the last element. Test a `for` loop with an empty collection, a singleton, and several elements. For a `while` loop, test a condition that is false initially, a single update, several updates, and each alternative stopping route. For recursion, test the base case, a near-base case, and a deeper case.

The total-load activity promises to sum every input load in newtons, with zero as the empty-list total.

Slide exercise: the omitted final load.

```python
# Deliberate defect: this loop omits the final measurement.
def total_load(loads_n):
    # A sum starts from its neutral value, zero newtons.
    total_n = 0.0
    # range excludes its stop; len(loads_n) - 1 stops too early.
    for index in range(len(loads_n) - 1):
        total_n += loads_n[index]
    return total_n

# Predict all three outputs before interpreting the results.
print(total_load([]))
print(total_load([5.0]))
print(total_load([2.0, 3.0, 4.0]))
```

Code 1.5: Slide exercise: the omitted final load

Example file: `ex05_total_load_bug.py`

```
0.0
0.0
5.0
```

For an empty list, `range(-1)` has no elements, so the initialized zero is returned. For a singleton, `range(0)` is also empty; the only load is never added. For three loads, `range(2)` supplies indices 0 and 1, omitting index 2. Therefore the program returns 5.0 N instead of 9.0 N for the last case.

Input	Expected (N)	Actual (N)	Result
`[]`	0.0	0.0	Pass
`[5.0]`	5.0	0.0	Fail
`[2.0, 3.0, 4.0]`	9.0	5.0	Fail

The narrow index-based repair is to use `range(len(loads_n))`. Because the index itself is unnecessary, the worked solution below expresses the contract even more directly by visiting every value.

Worked repair: include every load.

```python
# Worked repair using direct iteration instead of manual indexing.
def total_load(loads_n):
    # The empty input must still have total zero.
    total_n = 0.0
    # Visit every load once, including the last element.
    for load_n in loads_n:
        total_n += load_n
    return total_n

# These expectations are 0, 5, and 2 + 3 + 4 = 9 N.
print(total_load([]))
print(total_load([5.0]))
print(total_load([2.0, 3.0, 4.0]))
```

Code 1.6: Worked repair: include every load

Example file: `ex06_total_load_repair.py`

```
0.0
5.0
9.0
```

1.8. Recursive return paths need tests too

Lecture-slide alignment: Week 7 source PDF, p. 19. PDF page numbers include title and divider pages.

Week 6 introduced the factorial recurrence $0! = 1$ and $n! = n(n-1)!$ for positive integers. Week 7 reuses it to show that tests should exercise both direct termination and the return path of recursive calls.

Slide example: base and recursive paths.

```python
# Contract: n is a nonnegative integer; use small values here.
def factorial_recursive(n):
    # The base case stops immediately without another call.
    if n == 0:
        return 1
    # Reduce the input and use the returned smaller factorial.
    return n * factorial_recursive(n - 1)

# Test zero, one, and several recursive steps.
print(factorial_recursive(0))
print(factorial_recursive(1))
print(factorial_recursive(4))
```

Code 1.7: Slide example: base and recursive paths

Example file: `ex07_factorial_paths.py`

```
1
1
24
```

The input 0 reaches the base case with no recursive call. Input 1 makes one recursive call and

multiplies its returned 1 by 1. Input 4 reaches successive arguments 3, 2, 1, and 0; returned values combine to produce 24. These checks probe different execution histories, even though the program body is short.

This particular function assumes a nonnegative integer and does not reject invalid types or negative inputs. Testing that assumption is different from claiming it has been enforced. Keep inputs small enough for hand tracing and ordinary recursion depth. Comparing with an iterative factorial provides additional evidence, but the independent value $4! = 24$ remains important: two implementations can share the same wrong definition.

1.9. Floating-point agreement needs an explicit criterion

Lecture-slide alignment: Week 7 source PDF, p. 20. PDF page numbers include title and divider pages.

For many numerical results, ask whether an error is within a chosen tolerance instead of requiring exact equality. The familiar sum 0.1 + 0.2 illustrates the difference.

Slide example: exact equality versus tolerance.

```
# These decimal inputs are represented approximately in binary.
actual = 0.1 + 0.2
# The expected result is specified independently.
expected = 0.3
# This is a numerical-test tolerance, not a sensor specification.
absolute_tolerance = 1e-12

# Inspect the stored result and compare two acceptance rules.
print(actual)
print(actual == expected)
print(abs(actual - expected) <= absolute_tolerance)
```

Code 1.8: Slide example: exact equality versus tolerance

Example file: `ex08_absolute_tolerance.py`

```
0.30000000000000004
False
True
```

The absolute-error criterion is

$$|x - x_{\text{expected}}| \leq \varepsilon_{\text{abs}}.$$

The value `1e-12` means 10^{-12}; it is not a request to print twelve decimal places. If the compared values represent a physical quantity, this tolerance has that quantity's unit. An absolute tolerance for stress must be interpreted in pascals or megapascals consistently with the values being compared.

The supplied tolerance is a very small arithmetic-test threshold. It is not evidence that an instrument measures to that accuracy. Nor should a tolerance be increased until a genuinely incorrect model passes. Keep numerical approximation, measurement uncertainty, and physical-model error conceptually separate.

1.10. Engineering trends complement known answers

Lecture-slide alignment: Week 7 source PDF, p. 21. PDF page numbers include title and divider pages.

The model $\sigma(F, A) = F/A$, with $A > 0$, implies

$$\sigma(2F, A) = 2\sigma(F, A), \qquad \sigma(F, 2A) = \tfrac{1}{2}\sigma(F, A), \qquad \sigma(0, A) = 0.$$

A signed-force convention also requires the returned stress to preserve the sign of force. These relations provide useful sanity tests. For example, with $F = 10$ N and $A = 2$ m^2, the baseline is 5 Pa, doubled force gives 10 Pa, and doubled area gives 2.5 Pa.

However, a program returning $1000F/A$ would satisfy every scaling relation above while being wrong by a factor of 1000. Trend checks cannot replace an absolute known-answer check. This is why a convincing engineering test plan combines different kinds of evidence rather than relying on one attractive pattern.

1.11. Test the data and its preservation

Lecture-slide alignment: Week 7 source PDF, p. 22. PDF page numbers include title and divider pages.

Not every contractual promise concerns the final scalar. A function may promise to leave the original readings unchanged, retain the input order, preserve all required keys, or return one result per input. Those behaviors need tests of their own.

For lists, consider empty, singleton, and multiple readings; the first and last indices; and the original state after the call. Before a non-mutating calibration, save an independent copy of the flat numerical list. Compare the original with that copy afterward. Saving a second name that aliases the list is not independent evidence: both names would observe the same accidental mutation.

For dictionaries, test an existing key and a missing key separately. Check the correct nesting path and unit-bearing field names. A missing density is not a zero density. Testing a program that silently inserts zero may show repeatability while missing the more serious defect in the data policy.

1.12. Integration tests challenge the interfaces

Lecture-slide alignment: Week 7 source PDF, p. 23. PDF page numbers include title and divider pages.

A component can work correctly in isolation while receiving the wrong units from its caller. The slide diagram follows a complete chain: area in mm^2, area conversion to m^2, stress calculation in Pa, and reporting in MPa.

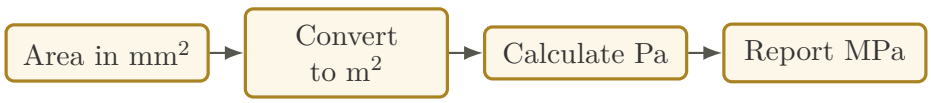


Figure 1.2.: The unit-interface chain from source page 23. Test the connections as well as the individual operations.

Worked example: connect unit conversion and stress.

```python
# Each small function has one input/output unit agreement.
def area_to_m2(area_mm2):
    # Squaring the millimetre-to-metre factor gives 1e-6.
    return area_mm2 / 1_000_000

def calculate_stress(force_n, area_m2):
    # Valid numerical inputs and positive area are assumed here.
    return force_n / area_m2

# Test the connected workflow, not merely the separate formulas.
area_m2 = area_to_m2(500.0)
stress_pa = calculate_stress(10000.0, area_m2)
stress_mpa = stress_pa / 1_000_000
# Independent benchmark: 10000 / 500 = 20 N/mm^2 = 20 MPa.
print(f"Area = {area_m2:.7f} m^2")
print(f"Stress = {stress_mpa:.2f} MPa")
```

Code 1.9: Worked example: connect unit conversion and stress

Example file: `ex10_integration_chain.py`

```
Area = 0.0005000 m^2
Stress = 20.00 MPa
```

The independent end-to-end benchmark is $10\,000/500 = 20$ N/mm^2 $= 20$ MPa. Passing 500 directly to a function expecting m^2 would still allow Python to divide two numbers, but would destroy the physical meaning of the result. An integration test detects this interface mismatch.

1.13. Build a small repeatable test driver

Lecture-slide alignment: Week 7 source PDF, p. 24. PDF page numbers include title and divider pages.

A **test driver** supplies cases, calls the function, compares results, and reports the evidence. The slides use a list of tuples: each tuple contains a temperature and its expected label. This reuses Week 5 collections without introducing a testing framework.

Slide example with required classifier definition.

```python
# Definition repeated here so this file runs independently.
def classify_temperature(temp_c, low_c, high_c):
    # Inputs are finite and limits are ordered, as in the slides.
    if temp_c < low_c:
        return "too low"
    elif temp_c <= high_c:
        return "normal"
    return "too high"

# Each tuple stores one input and its independently predicted label.
cases = [(19.9, "too low"), (20.0, "normal"),
         (30.0, "normal"), (30.1, "too high")]
# No selected tests have passed before any calls are made.
passed = 0
for temp_c, expected in cases:
    # Hold the two operating limits fixed across these tests.
    actual = classify_temperature(temp_c, 20.0, 30.0)
    # Compare the actual result with the prediction.
    ok = actual == expected
    print(temp_c, expected, actual, ok)
    if ok:
        passed += 1
# The qualifier "selected" prevents an unjustified universal claim.
print(f"Passed {passed}/{len(cases)} selected tests")
```

Code 1.10: Slide example with required classifier definition

Example file: `ex09_test_driver.py`

```
19.9 too low too low True
20.0 normal normal True
30.0 normal normal True
30.1 too high too high True
Passed 4/4 selected tests
```

At each pass, tuple unpacking binds `temp_c` and `expected`. The computed Boolean `ok` records whether that case agrees. The counter increments only for agreement. The final message says *selected tests*: passing four checks is not a claim that every possible input has been tested.

This printed driver is useful when learning, because a failed comparison remains visible without stopping later cases. Lecture 21 introduces assertions, which turn a failed expectation into an exception. Both approaches require good test inputs and independent expectations.

1.14. Worked activity: a five-case mean test plan

Lecture-slide alignment: Week 7 source PDF, p. 25. PDF page numbers include title and divider pages.

The contract states that `mean_reading(values)` returns the mean of a nonempty list of finite readings and does not modify its input. Empty input must be rejected. The source activity asks for a plan, not a function body; the following values are a worked answer.

Purpose	Input	Expected behavior
Known answer	`[2.0, 4.0, 6.0]`	Mean 4.0.
Singleton	`[7.0]`	Mean 7.0, not zero.
Equal readings	`[3.0, 3.0, 3.0]`	Mean 3.0.
Mixed signs	`[-2.0, 0.0, 2.0]`	Mean 0.0 through cancellation.
Empty input	`[]`	Rejection; no invented numerical mean.

For at least one valid case, save the input before calling the function and verify that it is unchanged afterward. The Lecture 19 contract requires rejection but does not yet identify an exception type. Lecture 21 makes that choice explicit with `ValueError`. Section 3.17 turns this plan into executable checks.

1.15. Keep a repaired defect as a regression case

Lecture-slide alignment: Week 7 source PDF, p. 26. PDF page numbers include title and divider pages.

A repair should leave behind evidence. First retain the smallest input that exposed the failure. Next write its expected outcome independently. Repair one cause, rerun the failing case, and then rerun the existing test suite. The broader rerun matters because a local change can alter another behavior.

Finally keep the new case in the suite. The FoS = 2.0 example is valuable long after the original operator has been changed: a later rewrite may accidentally restore the same boundary error. A defect without a retained test can become the same investigation again next week.

1.16. Lecture 19 exit ticket and worked answers

Lecture-slide alignment: Week 7 source PDF, p. 27. PDF page numbers include title and divider pages.

The new contract returns `"low"` below 100 kPa, `"normal"` from 100 to 200 kPa inclusive, and `"high"` above 200 kPa.

A typical normal case is 150 kPa. The exact-boundary tests are 100 and 200 kPa, both expected to be normal. These three cases alone do not enter the low or high branches. A glass-box inspection motivates adding values such as 99.9 and 200.1 kPa so those returns execute too.

Worked solution: pressure boundary tests.

```python
# This is the illustrative pressure rule in the exit ticket.
def classify_pressure(p):
    # p is a finite pressure expressed in kPa.
    if p < 100.0:
        return "low"
    elif p <= 200.0:
        return "normal"
    return "high"

# Include typical, exact-boundary, and outer-branch cases.
for pressure_kpa in [150.0, 100.0, 200.0, 99.9, 200.1]:
    print(pressure_kpa, classify_pressure(pressure_kpa))
```

Code 1.11: Worked solution: pressure boundary tests

Example file: `ex11_pressure_exit.py`

```
150.0 normal
100.0 normal
200.0 normal
99.9 low
200.1 high
```

Passing these tests supports the implemented classroom classification. It does not establish that 100 and 200 kPa are appropriate limits for a real apparatus, that the sensor is calibrated, or that other operating variables are acceptable.

Lecture 19 in one paragraph

Specify behavior before judging code. Select inputs deliberately, including equality and empty-data cases. Combine contract-based and path-based tests, check data preservation and unit interfaces, compare with independent answers, and retain defect-revealing cases. A passing suite supports a limited claim; it does not prove every implementation or engineering assumption correct.

2 Systematic Debugging and Error Messages

Lecture-slide alignment: Week 7 source PDF, pp. 28–51. PDF page numbers include title and divider pages.

Lecture 20. A defect is easier to repair when you can explain where the program first departs from its intended state. Debugging is not a contest to make an error message disappear. It is an investigation that links input, execution, observation, hypothesis, and repair.

Learning outcomes

Distinguish syntax, runtime, logic, unit, data, and numerical errors; read a traceback from symptom back to source; inspect values, types, units, and control flow; reduce a failing example and change one cause at a time; and turn a repair into documented regression evidence.

2.1. One short script can contain several defects

Lecture-slide alignment: Week 7 source PDF, pp. 29–30. PDF page numbers include title and divider pages.

The warm-up deliberately mixes a name mismatch with two unit defects. Its intended inputs are $F = 10\,000$ N, $A = 500$ mm^2, and a supplied yield strength of 250 MPa. Before running it, calculate a reference:

$$\sigma = \frac{10\,000}{500} = 20\,\mathrm{N/mm^2} = 20\,\mathrm{MPa}, \qquad \mathrm{FoS} = \frac{250}{20} = 12.5.$$

These independent expectations help diagnose the program after its immediate runtime failure has been removed.

Slide exercise: name mismatch and two unit defects.

```
# Deliberately faulty: do not use this as an engineering calculation.
Force = 10000.0  # Applied force in N; note the capital F.
area_mm2 = 500.0  # Area supplied in mm^2.
yield_strength_mpa = 250.0  # Supplied strength in MPa.

# Defect 1 in the arithmetic: this uses a length conversion for area.
area_m2 = area_mm2 / 1000
# Immediate runtime defect: force is not the name defined above.
stress_pa = force / area_m2
# Another defect: strength in MPa is mixed with stress in Pa.
fos = yield_strength_mpa / stress_pa
# These lines are not reached in a clean session.
print(f"Stress = {stress_pa:.2f} Pa")
print(f"FoS = {fos:.5f}")
```

Code 2.1: Slide exercise: name mismatch and two unit defects

Example file: `ex12_bar_original_bug.py`

In a clean interpreter, execution first fails at the reference to `force`. The defined name is `Force`; Python distinguishes letter case. The incorrect area conversion has already been evaluated, but

it does not raise an exception. The strength-to-stress ratio is also wrong, yet execution has not reached it. Fixing the first observed exception therefore does not imply that the remaining arithmetic is correct.

Predict the next observation

Repair only the inconsistent force name. Will the script now run? Will its area equal 0.0005 m^2? Predict the intermediate area, stress, and FoS before reading the staged repairs in Section 2.8.

2.2. Classify the symptom before choosing a repair

Lecture-slide alignment: Week 7 source PDF, p. 31. PDF page numbers include title and divider pages.

Different symptoms suggest different first actions. These categories describe the problem, not a rigid partition: a unit mistake may also cause a numerical overflow, and a data mistake may cause a lookup exception.

Category	What happened?	First useful action
Syntax	Python cannot parse the code.	Check punctuation, matching delimiters, and indentation.
Runtime	A reached statement raises an exception.	Read its traceback and inspect the supplied values.
Logic	Code runs but violates its specification.	Trace the first incorrect intermediate state or branch.
Unit	A quantity is interpreted in incompatible units.	Derive conversions and inspect interface units.
Data	A record has unexpected fields, meanings, or values.	Inspect the schema, origin, and relevant keys.
Numerical	An approximation fails its stated requirement.	Inspect tolerances, bounds, step size, and stopping evidence.

A result printed to six decimal places can still belong to any of the last four categories. Formatting is not evidence of a correct model, correct units, or a successful numerical method.

2.3. Syntax errors: inspect the structure

Lecture-slide alignment: Week 7 source PDF, p. 32. PDF page numbers include title and divider pages.

Python needs a valid grammatical structure before it can execute a script. In the slide example, the condition is understandable to a human, but the header lacks the colon that introduces the indented block.

Slide exercise: syntax error.

```
# Deliberately invalid Python: inspect the missing colon.
stress_mpa = 120.0
# Python cannot parse this if header as written.
if stress_mpa <= 125.0
    print("Within the stated limit")
```

Code 2.2: Slide exercise: syntax error

Example file: `ex13_missing_colon.py`

The repair is to place a colon after `125.0`. The comparison and the intended indentation need not change. Other common checks are paired parentheses, paired quotation marks, and consistent indentation of a block.

The location marked by Python identifies where parsing failed. It can lie after the actual cause: for example, an unmatched parenthesis on one line may make a later line appear unexpected. Read the nearby preceding code as well as the highlighted location. A syntax error in a script prevents that script from beginning normal execution; it is not a partly completed numerical calculation.

2.4. Runtime failures preserve a call trail

Lecture-slide alignment: Week 7 source PDF, pp. 33–34. PDF page numbers include title and divider pages.

A program can be grammatically valid and still attempt an invalid operation at runtime. The next example defines two functions successfully. Failure occurs only when `report()` calls the division with zero area.

Slide example: inspect the call trail.

```
# Deliberate runtime failure to make a traceback visible.
def calculate_stress(force_n, area_m2):
    # The caller supplies zero area, so this division fails.
    return force_n / area_m2

def report():
    # The bad argument originates here, one call before the symptom.
    print(calculate_stress(10000.0, 0.0))

# The call chain is module -> report -> calculate_stress.
report()
```

Code 2.3: Slide example: inspect the call trail

Example file: `ex14_traceback_demo.py`

The source slide shows this representative traceback. Path prefixes and version-dependent caret marks are omitted there, and its line numbers refer to the slide's shorter file, not to the annotated listing above.

```
Traceback (most recent call last):
  File "traceback_demo.py", line 8, in <module>
    report()
  File "traceback_demo.py", line 6, in report
    print(calculate_stress(10000.0, 0.0))
  File "traceback_demo.py", line 3, in calculate_stress
    return force_n / area_m2
ZeroDivisionError: float division by zero
```

Code 2.4: The abbreviated call trail printed on source page 34.

Read the bottom line first: the exception category is `ZeroDivisionError`, and the message identifies floating-point division by zero. Move upward to the failing operation, then to the caller that supplied the arguments. The original call is in the module, `report()` supplies the zero denominator, and `calculate_stress()` encounters it during arithmetic.

This separates **symptom location** from **cause**. The division line is where the inconsistency becomes impossible to continue, but a caller or an earlier data-processing stage may have introduced the value. A traceback is therefore a trail to investigate, not an instruction to edit the last line mechanically.

Engineering check

Changing the denominator from zero to an arbitrary tiny number would suppress this particular division failure while inventing an engineering answer. A proper repair follows the specified invalid-area policy, developed in Lecture 21.

2.5. Common exception names guide inspection

Lecture-slide alignment: Week 7 source PDF, p. 35. PDF page numbers include title and divider pages.

The last traceback line combines a category and details. Learn what the common categories suggest, but confirm the explanation using actual values and the failing operation.

Exception	Typical meaning here	Inspect first
`NameError`	A name is unavailable in the current context.	Spelling, letter case, scope, and definition/call order.
`TypeError`	An operation received an unsuitable type.	Actual value and type; a missing return that produced `None`.
`ValueError`	A value is unacceptable for an operation.	Text format or the explicitly checked valid range.
`IndexError`	A sequence position does not exist.	Index, length, and whether the list is empty.
`KeyError`	A requested dictionary key is absent.	Exact key, normalization policy, and nesting path.

For example, a `TypeError` in multiplication does not prove that multiplication itself is the root problem. A function called earlier may have printed a value and returned `None`. Similarly, a `KeyError` may refer to the material name or a property inside its record; inspect each lookup separately.

2.6. Debugging is a controlled investigation

Lecture-slide alignment: Week 7 source PDF, pp. 36–37. PDF page numbers include title and divider pages.

The slide workflow is: reproduce, locate and reduce, form a hypothesis, change one cause, rerun and compare, and retain a regression test. Each stage answers a specific question.

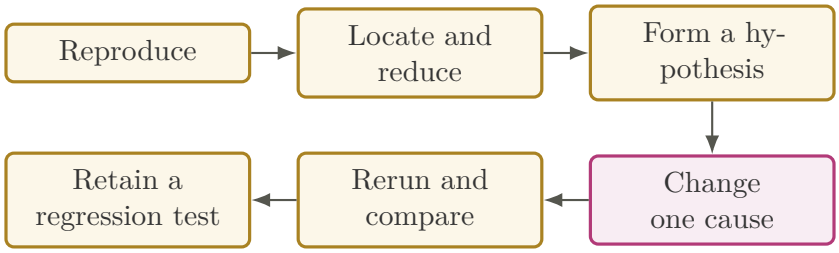


Figure 2.1.: The controlled debugging investigation from source page 36. A proposed change should predict a change in the observation.

Reproduce. Preserve the exact input, failing output, code version, and run order. State what should have happened and why. A vague report such as "it does not work" cannot distinguish a wrong output from a crash or a stalled loop.

Locate and reduce. Replace a large dataset by one record if that record still reproduces the same failure. Separate calculation from printing, and temporarily remove unrelated plotting or file saving. A **minimal reproducible example** is small enough to inspect completely but still demonstrates the defect. Removing the cause as well as the extra code would defeat the purpose.

Hypothesize and change. Predict the effect of one repair before applying it. For instance: "The area is too large because the conversion was not squared; using 10^{-6} should make it 0.0005 m^2." Randomly editing several lines may create a passing example without producing an explanation.

Compare and retain. Rerun the exact failing case, inspect the predicted intermediate value, and rerun related tests. Keep the smallest revealing case in the regression suite.

2.7. Inspect values, types, and invisible characters

Lecture-slide alignment: Week 7 source PDF, p. 38. PDF page numbers include title and divider pages.

Three simple tools support most beginner investigations. `print(value)` makes a value visible, `type(value)` identifies the kind of object, and `repr(text)` reveals a representation that includes quotes and visible escape sequences for otherwise hidden characters.

Slide example: inspect value, type, and whitespace.

```
# Preserve a raw field exactly as received for diagnosis.
area_text = " 500 "
# repr exposes quotes and surrounding whitespace.
print("Raw:", repr(area_text))
# type tells us that the raw field is still text.
print("Type:", type(area_text))
# Convert only after confirming that the field means area in mm^2.
area_mm2 = float(area_text)
print("Converted:", area_mm2, "mm^2")
```

Code 2.5: Slide example: inspect value, type, and whitespace

Example file: `ex15_inspect_text.py`

```
Raw: ' 500 '
Type: <class 'str'>
Converted: 500.0 mm^2
```

The raw value is text containing spaces, not yet a numerical area. The conversion succeeds because `float()` can parse the field. That success does not establish that the field actually means square millimetres; its physical interpretation must come from the data specification. Unit-bearing names and printed units help make that assumption inspectable.

Do not remove characters indiscriminately to make input parse. A leading zero in a component identifier may be meaningful. Use normalization only for a field whose specification permits it.

2.8. Repair the bar calculation one cause at a time

Lecture-slide alignment: Week 7 source PDF, pp. 39–42. PDF page numbers include title and divider pages.

2.8.1. Stage 1: repair the name, then inspect the next state

Using one consistent force name makes the script run, but retains its two unit mistakes.

Slide intermediate repair: unit defects remain.

```
# The name mismatch is repaired; the two unit defects remain.
force_n = 10000.0  # N.
area_mm2 = 500.0  # mm^2.
yield_strength_mpa = 250.0  # MPa.

# Deliberate area-conversion defect: inspect this intermediate value.
area_m2 = area_mm2 / 1000
stress_pa = force_n / area_m2
# Deliberate incompatible units in the ratio below.
fos = yield_strength_mpa / stress_pa
# A script running to completion is not evidence of correct units.
print("Area:", area_m2, "m^2")
print("Stress:", stress_pa, "Pa")
print("FoS:", fos)
```

Code 2.6: Slide intermediate repair: unit defects remain

Example file: `ex16_bar_name_repaired.py`

```
Area: 0.5 m^2
Stress: 20000.0 Pa
FoS: 0.0125
```

The first incorrect intermediate value is `area_m2 = 0.5`. That is not the conversion of 500 mm^2. Debugging from the final FoS alone is harder because it combines several operations; the intermediate area reveals the earlier defect directly.

2.8.2. Stage 2: derive the area conversion

Length converts as $1\,\text{mm} = 10^{-3}\,\text{m}$. Squaring both sides gives

$$1\,\text{mm}^2 = 10^{-6}\,\text{m}^2, \qquad 500\,\text{mm}^2 = 0.0005\,\text{m}^2.$$

The corrected stress is $10\,000/0.0005 = 20 \times 10^6$ Pa. The next listing is a worked intermediate stage: it makes the slide's area repair but deliberately leaves the strength-unit problem.

Worked intermediate stage: area conversion repaired.

```python
# Intermediate repair of the slide exercise: one unit defect remains.
force_n = 10000.0  # N.
area_mm2 = 500.0  # mm^2.
yield_strength_mpa = 250.0  # MPa.

# Correct: 1 mm^2 = 1e-6 m^2.
area_m2 = area_mm2 / 1_000_000
stress_pa = force_n / area_m2
# Still wrong: MPa and Pa are not the same numerical unit.
fos = yield_strength_mpa / stress_pa
print("Area:", area_m2, "m^2")
print("Stress:", stress_pa, "Pa")
print("FoS:", fos)
```

Code 2.7: Worked intermediate stage: area conversion repaired

Example file: `ex17_bar_area_repaired.py`

```
Area: 0.0005 m^2
Stress: 20000000.0 Pa
FoS: 1.25e-05
```

The observed FoS becomes 1.25×10^{-5}, not 12.5, because the numerator is still numerically in MPa and the denominator in Pa. A ratio of stresses is dimensionless only after the numerical units are made consistent.

2.8.3. Stage 3: align units before taking the ratio

Slide example: consistent internal SI units.

```python
# Inputs retain their stated units in their names.
force_n = 10000.0
area_mm2 = 500.0

yield_strength_mpa = 250.0

# Convert area and strength before the two calculations.
area_m2 = area_mm2 / 1_000_000
strength_pa = yield_strength_mpa * 1_000_000
# Stress has unit Pa; the ratio of like stress units is dimensionless.
stress_pa = force_n / area_m2
fos = strength_pa / stress_pa
# Convert for reporting, while leaving the stored stress in Pa.
print(f"Stress = {stress_pa / 1e6:.2f} MPa")
print(f"Factor of safety = {fos:.2f}")
```

Code 2.8: Slide example: consistent internal SI units

Example file: `ex18_bar_complete_repair.py`

```
Stress = 20.00 MPa
Factor of safety = 12.50
```

The conversion to pascals multiplies 250 MPa by 10^6. Both strength and stress now have the

same unit, so their ratio is 12.5. The final print converts stress to MPa for readability without changing the stored internal value.

An independent route is even shorter: $1\,\mathrm{N/mm^2} = 1\,\mathrm{MPa}$, so $10\,000/500 = 20$ MPa and $250/20 = 12.5$. Agreement with this separately derived benchmark supports the repaired unit chain.

Observation	Hypothesis / repair	New evidence
`NameError`	Use a consistent force name.	Execution reaches the calculation.
Area = 0.5 $\mathrm{m^2}$	Square the length conversion.	Area = 0.0005 $\mathrm{m^2}$.
Implausible FoS	Convert strength and stress to the same unit.	Dimensionless FoS = 12.5.
Risk of recurrence	Retain independent end-to-end tests.	20 MPa and 12.5 are checked again.

Engineering check

This calculation assumes uniform axial stress and uses the supplied material strength. It does not assess buckling, fatigue, stress concentrations, other load cases, or the validity of the material input. The slide's limited claim must remain visible in a report.

2.9. Trace a state variable instead of only its final value

Lecture-slide alignment: Week 7 source PDF, p. 43. PDF page numbers include title and divider pages.

The energy example supplies three power readings, each applied for half an hour. The independent expected total is $(1 + 2 + 3) \times 0.5 = 3$ kWh. The faulty implementation resets its accumulator inside the loop.

Slide exercise: accumulator reset inside loop.

```python
# Deliberate logic defect: the running total is reset every pass.
powers_kw = [1.0, 2.0, 3.0]  # Piecewise constant power in kW.
interval_h = 0.5  # Duration of each recorded interval in hours.
for power_kw in powers_kw:
    # This initialization is in the wrong block.
    total_kwh = 0.0
    total_kwh += power_kw * interval_h
    print("Intermediate:", total_kwh)
# Only the final interval survives in this incorrect total.
print("Final:", total_kwh, "kWh")
```

Code 2.9: Slide exercise: accumulator reset inside loop

Example file: `ex19_accumulator_bug.py`

```
Intermediate: 0.5
Intermediate: 1.0
Intermediate: 1.5
Final: 1.5 kWh
```

Power (kW)	Added (kWh)	Faulty total (kWh)	Required running total (kWh)
1.0	0.5	0.5	0.5
2.0	1.0	1.0	1.5
3.0	1.5	1.5	3.0

The first pass appears correct. The divergence begins at the second pass, when the prior 0.5 kWh is discarded. Move the initialization before the loop so that it executes once.

Worked repair: preserve the running total.

```
# The input units make each product an energy in kWh.
powers_kw = [1.0, 2.0, 3.0]
interval_h = 0.5
# Initialize once; this also defines the empty-list result.
total_kwh = 0.0
for power_kw in powers_kw:
    # Add this interval without discarding earlier intervals.
    total_kwh += power_kw * interval_h
    print("Intermediate:", total_kwh)
print("Final:", total_kwh, "kWh")
```

Code 2.10: Worked repair: preserve the running total

Example file: `ex20_accumulator_repair.py`

```
Intermediate: 0.5
Intermediate: 1.5
Intermediate: 3.0
Final: 3.0 kWh
```

An empty input also exposes the original design problem. In a clean top-level script, the loop would never assign `total_kwh`, and the final print would raise `NameError`. With initialization outside the loop, the empty-list total is a defined zero. Inside a function, an analogous unassigned local name can instead produce `UnboundLocalError`; the important cause is the skipped initialization.

2.10. Trace an unexpected None to the function that returned it

Lecture-slide alignment: Week 7 source PDF, p. 44. PDF page numbers include title and divider pages.

Slide exercise: printed current but no return.

```
# Deliberate defect: printing does not return the calculated current.
def calculate_current(voltage_v, resistance_ohm):
    current_a = voltage_v / resistance_ohm
    print(current_a)  # A human sees the number; the caller gets None.

# This prints 3.0 from inside the function, but stores None.
current_a = calculate_current(12.0, 4.0)
print("Stored:", current_a)
# A number cannot be multiplied by None; this raises TypeError.
power_w = 12.0 * current_a
```

Code 2.11: Slide exercise: printed current but no return

Example file: `ex21_missing_return.py`

The function computes 3.0 A and prints it. However, it reaches the end without a value-returning statement, so the caller receives `None`. The output before failure is therefore a line containing 3.0 followed by `Stored: None`. Multiplying 12.0 by `None` then raises `TypeError`.

The failing multiplication is downstream of the actual defect. The minimal repair is to return the current from the calculation function and print it in the reporting layer.

Worked repair: return before reporting.

```
# Calculation receives V and ohm, and returns current in A.
def calculate_current(voltage_v, resistance_ohm):
    # Positive resistance is an input assumption for this example.
    current_a = voltage_v / resistance_ohm
    return current_a

# The caller receives a reusable numerical value.
current_a = calculate_current(12.0, 4.0)
power_w = 12.0 * current_a
# Reporting is separate from calculation.
print("Current:", current_a, "A")
print("Power:", power_w, "W")
```

Code 2.12: Worked repair: return before reporting

Example file: `ex22_current_repair.py`

```
Current: 3.0 A
Power: 36.0 W
```

Check the returned value, not merely whether a number appeared on screen. This example directly connects Week 4 function interfaces to Week 7 fault diagnosis.

2.11. Debug side effects with a saved-state test

Lecture-slide alignment: Week 7 source PDF, p. 45. PDF page numbers include title and divider pages.

The calibration function promises not to modify raw temperatures. Its arithmetic can return the expected corrected values while violating that second promise.

Slide exercise: unintended change to raw data.

```python
# Deliberate violation of the promised non-mutation contract.
def corrected_readings(raw_c, offset_c):
    # Assignment creates an alias, not a new list.
    processed_c = raw_c
    for i in range(len(processed_c)):
        # This also changes the caller's raw_c list.
        processed_c[i] -= offset_c
    return processed_c

# Save the given readings, then observe both names after the call.
raw_c = [22.0, 23.0]
result_c = corrected_readings(raw_c, 0.5)
print("Raw:", raw_c)
print("Processed:", result_c)
```

Code 2.13: Slide exercise: unintended change to raw data

Example file: `ex23_alias_bug.py`

```
Raw: [21.5, 22.5]
Processed: [21.5, 22.5]
```

The line `processed_c = raw_c` binds two names to the same list. Each indexed update therefore changes the caller's original data as well as the returned list. A test that checks only the processed numbers would miss the violation.

Worked repair: copy and test the saved state.

```python
# Return offset-corrected temperatures without changing raw data.
def corrected_readings(raw_c, offset_c):
    # A shallow list copy is sufficient for this flat list of numbers.
    processed_c = raw_c.copy()
    for i in range(len(processed_c)):
        processed_c[i] -= offset_c
    return processed_c

raw_c = [22.0, 23.0]
# Preserve independent evidence of the pre-call state.
before = raw_c.copy()
result_c = corrected_readings(raw_c, 0.5)
# Test both the answer and the non-mutation promise.
assert result_c == [21.5, 22.5]
assert raw_c == before
print("Raw:", raw_c)
print("Processed:", result_c)
```

Code 2.14: Worked repair: copy and test the saved state

Example file: `ex24_alias_repair.py`

```
Raw: [22.0, 23.0]
Processed: [21.5, 22.5]
```

The repair creates a new outer list. That shallow copy is sufficient here because each element is an immutable number. It does not promise independent copies of arbitrary nested mutable

records. The saved `before` list is also a copy, so the non-mutation assertion compares against real pre-call evidence rather than another alias.

2.12. Debug dictionary lookups one level at a time

Lecture-slide alignment: Week 7 source PDF, p. 46. PDF page numbers include title and divider pages.

Slide exercise: material-name lookup mismatch.

```python
# Illustrative property data, not a certified materials database.
materials = {"steel": {"density_kg_m3": 7850.0}}
requested = " Steel "
# Inspect the exact requested key and the available keys.
print("Requested:", repr(requested))
print("Known names:", list(materials.keys()))
# The unnormalized key does not exist, so this raises KeyError.
record = materials[requested]
# This line is not reached after the failed first lookup.
print(record["density_kg_m3"])
```

Code 2.15: Slide exercise: material-name lookup mismatch

Example file: `ex25_dictionary_bug.py`

The first printed field reveals `' Steel '`, while the available outer key is `'steel'`. The first lookup raises `KeyError`; the property lookup is never reached. The diagnosis is not "density is missing" until the material record has actually been obtained.

In this controlled material-name field, the intended normalization permits stripping surrounding whitespace and ignoring capitalization. The worked repair retains the original request for diagnosis and checks the two lookups separately.

Worked repair: normalize the permitted name field.

```python
# This controlled material-name field is case-insensitive by policy.
materials = {"steel": {"density_kg_m3": 7850.0}}
requested = " Steel "
# Preserve requested while constructing a normalized lookup key.
key = requested.strip().lower()
# Inspect the outer lookup separately from the property lookup.
if key not in materials:
    raise KeyError(f"Unknown material: {requested!r}")
record = materials[key]
# Do not replace missing density with zero.
if "density_kg_m3" not in record:
    raise KeyError("Required density_kg_m3 is missing")
print(record["density_kg_m3"])
```

Code 2.16: Worked repair: normalize the permitted name field

Example file: `ex26_dictionary_repair.py`

```
7850.0
```

The density is illustrative slide data, not a certified material specification. More importantly, replacing an absent density with 0.0 would fabricate a measurement and conceal a defect. A

valid property value and a missing property are different states that a test must distinguish.

2.13. Nontermination and hidden state need evidence too

Lecture-slide alignment: Week 7 source PDF, pp. 47–48. PDF page numbers include title and divider pages.

When a loop does not finish, identify the variable in its stopping condition and trace two or three passes. Does that variable change? Does the update approach a reachable stopping state? For numerical searches, inspect the tolerance, bounds, and work budget as well as the arithmetic.

A maximum-iteration limit is a work safeguard, not evidence that the last iterate is a solution. After the loop, separately check the acceptance criterion. A correct failure report is preferable to presenting an unaccepted candidate as an answer.

Clean-session testing. A notebook can retain names defined by earlier cells. The misspelled-force example might appear to run because an old value of `force` remains in memory, creating a more dangerous silent error. Before sharing, restart the interpreter or notebook kernel and execute the document from the beginning. Every required input and definition should appear in the saved code in the correct order.

Evidence to report

A peer should be able to reproduce the result from the delivered files, not from your previous interactive session. Record input, code version, execution order, and the observed failure or result.

2.14. Worked activity: diagnose an entire tiny program

Lecture-slide alignment: Week 7 source PDF, p. 49. PDF page numbers include title and divider pages.

Reconstructed exercise, not a recovered attachment

The source slide names `debug_lab_starter.py` and specifies three defects: a returned-value defect, a unit-conversion defect, and an inclusive-boundary defect. It gives a target benchmark of 125 MPa and FoS = 2.0, but does not show the file's contents. The following new starter uses 10 000 N, 80 mm^2, and 250 MPa to realize that benchmark. These chosen inputs and implementation are a reading-note reconstruction.

Reconstructed activity starter, not recovered source.

```python
# Reconstruction from the defect descriptions on source PDF page 49.
# The PDF names a starter file but does not display that file's code.
def area_to_m2(area_mm2):
    # Deliberate unit bug: use the squared conversion in the repair.
    return area_mm2 / 1000

def calculate_stress(force_n, area_m2):
    stress_pa = force_n / area_m2
    # Deliberate returned-value defect: should return the stress.
    print(stress_pa)

def classify_design(fos):
    # Deliberate inclusive-boundary defect: exactly 2.0 should pass.
    if fos > 2.0:
        return "acceptable"
    return "not acceptable"

# Chosen reconstruction inputs give the slide's 125 MPa, FoS=2 goal.
area_m2 = area_to_m2(80.0)
stress_pa = calculate_stress(10000.0, area_m2)
# None from the faulty function causes the immediate TypeError here.
fos = 250e6 / stress_pa
print(stress_pa / 1e6, classify_design(fos))
```

Code 2.17: Reconstructed activity starter, not recovered source

Example file: `ex27_reconstructed_lab_starter.py`

First, the stress function prints 125000.0 Pa but returns `None`, producing a `TypeError` when the ratio is attempted. Replacing that print with a return allows execution to continue but does not repair the area conversion. With the wrong conversion, the stress is 0.125 MPa rather than 125 MPa. Fixing the squared conversion yields the benchmark FoS = 2.0, which then exposes the final strict-inequality defect.

Each cause deserves its own retained test: one area-conversion test, one returned-stress test, and one exact-boundary classification test. An end-to-end check then confirms that the repaired components connect correctly.

Worked repair of the reconstructed starter.

```python
# This is a solution to the reconstructed activity, not a source file.
def area_to_m2(area_mm2):
    # Correct the squared length-unit conversion.
    return area_mm2 / 1_000_000

def calculate_stress(force_n, area_m2):
    # Return the result rather than merely printing it.
    return force_n / area_m2

def classify_design(fos):
    # The stated rule accepts equality at 2.0.
    if fos >= 2.0:
        return "acceptable"
    return "not acceptable"

# Retain a regression test associated with each repaired cause.
assert abs(area_to_m2(80.0) - 0.00008) <= 1e-15
assert calculate_stress(10.0, 2.0) == 5.0
assert classify_design(2.0) == "acceptable"
# Test the connected workflow against the independent benchmark.
stress_pa = calculate_stress(10000.0, area_to_m2(80.0))
fos = 250e6 / stress_pa
assert abs(stress_pa / 1e6 - 125.0) <= 1e-9
assert abs(fos - 2.0) <= 1e-12
print(f"Stress = {stress_pa / 1e6:.2f} MPa")
print(f"FoS = {fos:.2f}; {classify_design(fos)}")
```

Code 2.18: Worked repair of the reconstructed starter

Example file: `ex28_reconstructed_lab_solution.py`

```
Stress = 125.00 MPa
FoS = 2.00; acceptable
```

The order of repair explains why a program may appear to become “more broken” when an earlier error is fixed: a previously unreached line is now executed and reveals another defect. That is progress in the investigation, not evidence that the first repair was necessarily wrong.

2.15. Ask for useful help without transferring responsibility

Lecture-slide alignment: Week 7 source PDF, p. 50. PDF page numbers include title and divider pages.

A useful help request supplies a minimal reproducible example, the exact exception or unexpected output, expected behavior, input units, and the hypothesis already tested. This applies to a classmate, an instructor, documentation, or an AI assistant.

For example: “For 10 000 N and 500 mm^2, I expect 20 MPa by hand calculation. The program runs but prints 0.02 MPa. I suspect the area conversion. Please explain the first wrong intermediate value rather than rewrite the whole program.” This is a worked phrasing of the slide’s evidence-based help principle, not an additional required submission format.

Remove personal data, confidential records, and credentials before sharing code or output. Request an explanation, inspect every changed line, and retest the original failure. Follow the actual permitted-assistance rules; these notes do not grant new permission to use a tool in

assessed work.

2.16. Lecture 20 exit ticket and worked answers

Lecture-slide alignment: Week 7 source PDF, p. 51. PDF page numbers include title and divider pages.

What is on the last traceback line? Usually the exception type and a message describing the immediate problem. In the stress demonstration, these are `ZeroDivisionError` and floating-point division by zero.

Why can the cause occur earlier? A caller may supply an invalid value, an earlier conversion may use the wrong unit, or a function may return `None`. The final failing operation only reveals the inconsistency.

Explain one bar-calculation unit defect. Millimetres to metres uses 10^{-3}, but square millimetres to square metres uses 10^{-6}. Another defect is taking a numerical ratio between MPa and Pa without conversion.

What follows one passing repaired example? Rerun related cases and the existing suite, retain a regression case for the repaired defect, and verify the delivered program in a clean session.

Lecture 20 in one paragraph

Debugging replaces random edits with a reproducible explanation of cause and effect. Read the symptom, reduce the example, inspect values and meaning, predict the effect of one repair, and test that prediction. Do not confuse a removed exception with a corrected engineering result.

3 Exceptions, Assertions, and Reliable Programs

Lecture-slide alignment: Week 7 source PDF, pp. 52–77. PDF page numbers include title and divider pages.

Lecture 21. The central question is: *How should the program fail when it cannot produce a justified result?* A reliable program does not manufacture an answer merely to continue execution. It identifies invalid input, reports or handles expected failures, and keeps test expectations executable.

Learning outcomes

Raise an exception for a violated input contract; handle expected failures with narrow `try/except` blocks; trace normal and exceptional execution; use assertions for tests and internal expectations; and combine validation, computation, and evidence in a small engineering report.

3.1. What should a zero-area request produce?

Lecture-slide alignment: Week 7 source PDF, pp. 53–55. PDF page numbers include title and divider pages.

The warm-up asks what an axial-stress function should do with $F = 100$ N and $A = 0$ m^2. Returning zero stress would be a false calculation. Returning an arbitrary huge number is also unjustified. Returning `None` can be a valid documented convention, but every caller must recognize it and avoid using it as a number. In this lecture, an explicit exception makes the invalid-area path clear.

Definition

An **exception** interrupts the current normal execution path. Control transfers to a matching handler if one is available; otherwise the exception propagates outward and may terminate the run with a traceback. A handler does not automatically resume execution at the failed operation.

The normal path accepts valid inputs, calculates a quantity, and returns it. The exceptional path communicates why no valid result is available. The goal is not to catch every possible error. Handle expected failures that can be explained or recovered from; leave unexpected defects visible for diagnosis.

3.2. Raise an error before invalid arithmetic

Lecture-slide alignment: Week 7 source PDF, p. 56. PDF page numbers include title and divider pages.

The initial validated function preserves the earlier formula but explicitly rejects nonpositive area.

Slide example: explicit invalid-area rejection.

```python
# Core scope: finite numerical inputs; signed force is allowed.
def calculate_stress(force_n, area_m2):
    """Return uniform axial stress in Pa from N and m^2."""
    # Reject an invalid denominator before the division is attempted.
    if area_m2 <= 0:
        raise ValueError("Area must be positive, in m^2.")
    # This line is reached only when the area check has passed.
    return force_n / area_m2

# The known positive-area example still returns five pascals.
print(calculate_stress(10.0, 2.0))
```

Code 3.1: Slide example: explicit invalid-area rejection

Example file: `ex29_validated_stress.py`

```
5.0
```

The docstring states the quantity and units. The condition `area_m2 <= 0` identifies a value that violates the denominator requirement. The `raise` statement constructs a `ValueError` with a meaningful message and leaves normal execution of that call. The division is reached only after the area check has passed.

For zero area, the failure is now deliberate rather than an accidental `ZeroDivisionError`. Negative area is rejected too, even though Python could divide by it. Signed or zero force is still allowed by this function's stated model.

An enforced check is not the same as an assumed precondition

The source explicitly assumes finite numerical inputs at this stage. This function checks positive area, not every possible input type or numerical value. It does not by itself reject all NaNs, infinities, Booleans, or inappropriate objects. The later integrated example adds explicit checks rather than silently changing this source example.

3.3. Choose an exception that explains the failure

Lecture-slide alignment: Week 7 source PDF, p. 57. PDF page numbers include title and divider pages.

Choose a built-in exception category that communicates what went wrong. In the slide examples, `TypeError` indicates an unsuitable kind of value, `ValueError` an unacceptable value, `KeyError` a missing required dictionary field, and `AssertionError` a failed test or internal claim.

For example, a list where an ordinary numerical force is required is a type problem; a zero area is a value problem; a material record without `density_kg_m3` has a missing-field problem. A useful error message identifies the quantity, the required condition, and the expected unit. "Area must be positive, in m^2" helps more than "Bad input."

The category is a design choice tied to the function's contract. It should be tested and documented consistently so callers can handle it deliberately. No custom exception class is needed for these exercises.

3.4. Handle an expected conversion error

Lecture-slide alignment: Week 7 source PDF, pp. 58–59. PDF page numbers include title and divider pages.

The `try` clause contains the protected operation. A matching `except` clause describes what to do if that operation raises a particular exception. This example uses a fixed text field so that no keyboard input is needed.

Slide example: a handled conversion failure.

```
# Fixed text simulates input without asking for keyboard interaction.
text = "not a number"
try:
    # This conversion raises ValueError for the supplied text.
    force_n = float(text)
    # Skipped when conversion fails; force_n was not assigned.
    print("Converted force:", force_n)
except ValueError:
    # Handle this expected format failure, not all exceptions.
    print("Force must be a numerical value in N.")
# Execution resumes here after the matching handler completes.
print("Input check finished")
```

Code 3.2: Slide example: a handled conversion failure

Example file: `ex30_catch_conversion.py`

```
Force must be a numerical value in N.
Input check finished
```

Python reaches the conversion and raises `ValueError`. It skips the rest of the `try` block, including the print of `force_n`. The matching handler prints an explanation. Execution then continues at the final, unindented print. The failed assignment did not create a valid new force value, so the handler should not try to print one.

If the input were `"12.5"`, conversion would succeed, the print inside `try` would execute, the handler would be skipped, and the final print would still execute. If a different exception occurred without a matching handler, it would propagate outward rather than being treated as a conversion error.

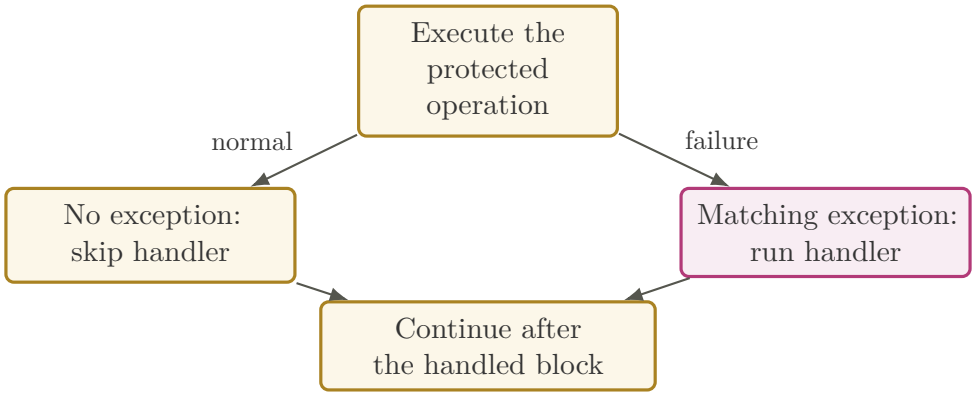


Figure 3.1.: The two routes in source page 59. An unmatched exception propagates outward; neither route returns automatically to the failed line.

3.5. Worked trace: which letters are printed?

Lecture-slide alignment: Week 7 source PDF, p. 60. PDF page numbers include title and divider pages.

Slide activity: trace a handled failure.

```
# Change only this input to "12.5" to trace the successful route.
text = "bad"
try:
    print("A")  # Reached before attempting the conversion.
    value = float(text)  # A bad field transfers control to except.
    print("B")  # Reached only when conversion succeeds.
except ValueError:
    print("C")  # Reached only on this matching conversion failure.
print("D")  # Reached after either ordinary or handled execution.
```

Code 3.3: Slide activity: trace a handled failure

Example file: `ex31_exception_letters.py`

```
A
C
D
```

For `text = "bad"`, A is printed before conversion, B is skipped because conversion fails, C is printed by the handler, and D is printed after the block. For `text = "12.5"`, the sequence is A, B, D. The handler's C is skipped because no matching failure occurred.

Input	Printed sequence	Reason
`"bad"`	A, C, D	Conversion transfers control to the matching handler.
`"12.5"`	A, B, D	Conversion completes; the handler is not used.

The location of D matters. It is outside both clauses, so either a normal run or this handled failure reaches it. An unhandled exception would prevent that ordinary continuation.

3.6. Different failures can require different responses

Lecture-slide alignment: Week 7 source PDF, p. 61. PDF page numbers include title and divider pages.

A protected block can distinguish more than one expected exception. The resistance example separates a malformed numerical field from division by zero.

Slide example: distinguish conversion and division.

```python
# Try "bad", "0", and "4" as three separate input experiments.
text = "0"
try:
    # A malformed string raises ValueError at this line.
    resistance_ohm = float(text)
    # Zero resistance instead raises ZeroDivisionError here.
    current_a = 12.0 / resistance_ohm
except ValueError:
    print("Resistance must be numerical.")
except ZeroDivisionError:
    print("Resistance cannot be zero.")
else:
    # Only the successful try route has a calculated current to print.
    print(f"Current = {current_a:.2f} A")
```

Code 3.4: Slide example: distinguish conversion and division

Example file: `ex32_multiple_handlers.py`

```
Resistance cannot be zero.
```

For `"bad"`, parsing fails and the first handler reports that resistance must be numerical. For `"0"`, parsing succeeds but division fails, so the second handler reports that resistance cannot be zero. For `"4"`, both operations succeed and the current is 3.00 A.

Only the matching handler executes. The final `else` is the successful route, not a catch-all error handler. This example is intentionally a control-flow demonstration: it does not implement a complete positive, finite resistance contract. In particular, a negative resistance would not produce a division-by-zero exception, and a parseable nonfinite value would require a separate validation policy.

3.7. Use else to isolate successful follow-up

Lecture-slide alignment: Week 7 source PDF, p. 62. PDF page numbers include title and divider pages.

The `else` associated with `try/except` runs after the `try` suite completes normally, with no exception and no early exit such as a return or loop-control transfer. In the simple examples here, that means the conversion has succeeded.

Slide example: keep the protected operation narrow.

```
# A numerical field allows the successful route to be observed.
text = "12.5"
try:
    # Protect only the operation whose failure we intend to handle.
    force_n = float(text)
except ValueError:
    print("Force must be numerical.")
else:
    # Errors in this block are not caught by the preceding handler.
    print(f"Accepted field: {force_n:.1f} N")
```

Code 3.5: Slide example: keep the protected operation narrow

Example file: `ex33_else_success.py`

```
Accepted field: 12.5 N
```

Moving successful reporting outside the protected conversion block reduces accidental handling of an unrelated later defect. If the `else` block itself raises an exception, the preceding `except` clause does not catch it. The same principle applies to an exception raised inside a handler: a neighboring handler on that same `try` does not automatically take over.

This elaborates the distinction already emphasized in the slide. Keep a `try` block narrow enough that the associated handler has a specific, credible interpretation of the failure.

3.8. A caller can handle a function's rejection

Lecture-slide alignment: Week 7 source PDF, p. 63. PDF page numbers include title and divider pages.

A handler can catch an exception raised inside a function called by its `try` block. This lets the calculation function reject invalid inputs while the caller decides how to communicate or recover.

Slide example with its required function.

```python
# Definition repeated so that this example is self-contained.
def calculate_stress(force_n, area_m2):
    # Inputs are finite numbers; this explicitly checks positive area.
    if area_m2 <= 0:
        raise ValueError("Area must be positive, in m^2.")
    return force_n / area_m2

try:
    # The function, not the caller's print statement, rejects the area.
    stress_pa = calculate_stress(100.0, 0.0)
except ValueError as error:
    # Preserve the explanation attached to the exception object.
    print(f"Cannot calculate stress: {error}")
else:
    # No stress is printed on the exceptional route.
    print(f"Stress = {stress_pa:.2f} Pa")
```

Code 3.6: Slide example with its required function

Example file: `ex34_caller_handles_error.py`

```
Cannot calculate stress: Area must be positive, in m^2.
```

The phrase `as error` binds the caught exception object to a name inside the handler. In the f-string, its text communicates the original explanation. The function has one responsibility: calculate stress for valid input or reject invalid area. The caller has another: report that rejection, retry, or stop.

The print in `else` is not reached on the invalid-area route. There is therefore no numerical stress or PASS label after failure. Preserving this separation prevents stale or undefined results from being presented as fresh calculations.

3.9. Combine a retry loop with a deliberate exit

Lecture-slide alignment: Week 7 source PDF, p. 64. PDF page numbers include title and divider pages.

The retry example combines Week 2 control flow with expected-error handling. It has three broad outcomes: cancel, reject and retry, or accept and stop.

Slide example: retry, reject, or cancel.

```
# This initial slide example assumes finite numeric input after parsing.
while True:
    # Remove surrounding spaces; retain the text for the q check.
    text = input("Area in m^2 (or q): ").strip()
    if text.lower() == "q":
        print("Cancelled")
        break  # Leave the loop deliberately without a result.
    try:
        area_m2 = float(text)  # A malformed number raises ValueError.
    except ValueError:
        print("Enter a number.")
        continue  # Start a new pass and ask again.
    if area_m2 <= 0:
        print("Area must be positive.")
        continue  # Do not accept a nonpositive area.
    print("Accepted:", area_m2, "m^2")
    break  # Stop after an accepted reading.
```

Code 3.7: Slide example: retry, reject, or cancel

Example file: `ex35_retry_area.py`

```
Area in m^2 (or q): bad
Enter a number.
Area in m^2 (or q): 0
Area must be positive.
Area in m^2 (or q): 0.002
Accepted: 0.002 m^2
```

The sample interaction supplies `bad`, then `0`, then `0.002`. The first field fails parsing. The second parses but violates the positive-area requirement. The third is accepted. The shown transcript includes the simulated typed inputs for readability.

A `continue` begins the next loop pass; it does not merely continue at the next line below the handler. A `break` leaves the loop. Checking `q` before numerical conversion gives the user an intentional nonnumerical exit. Stripping whitespace and normalizing the cancellation token makes `" Q "` a cancellation too.

Preserve the source's preview limitation

Source page 64 notes that its range check assumes a finite number. `float("nan")` and `float("inf")` can succeed, and the positive-range check alone is not a complete finite-data validator. The original example is retained here. A labelled extension combining it with the finiteness tool appears in Section 4.4.

3.10. Never turn every failure into plausible data

Lecture-slide alignment: Week 7 source PDF, p. 65. PDF page numbers include title and divider pages.

The slide counterexample uses a broad handler to return zero whenever its division raises an ordinary exception.

Slide counterexample: broad handler fabricates zero.

```python
# Deliberately bad policy: do not use this as a repaired calculation.
def hidden_failure(force_n, area_m2):
    try:
        return force_n / area_m2
    except Exception:
        # This disguises a failed calculation as a numerical result.
        return 0.0

# Division by zero is hidden, so the observation is misleading.
print(hidden_failure(100.0, 0.0))
```

Code 3.8: Slide counterexample: broad handler fabricates zero

Example file: `ex36_hidden_failure.py`

```
0.0
```

For a nonzero force and zero area, 0.0 Pa is not the calculated stress. It is a fabricated replacement that could be interpreted as an unloaded component or a reassuring result. Catching more categories does not make this design more reliable; it hides more causes.

A better policy is to validate the domain explicitly, catch only failures that can be explained or recovered from, and allow unexpected defects to remain visible. A broad `except Exception` is not prohibited in every conceivable program, but the *silent replacement with zero* is the key defect in this example.

3.11. Assertions make expectations executable

Lecture-slide alignment: Week 7 source PDF, p. 66. PDF page numbers include title and divider pages.

An **assertion** records a claim that should hold during testing or development. With assertions active, a true condition allows execution to continue; a false condition raises `AssertionError`, optionally with an explanatory message.

Slide example: an executable expectation.

```python
# The function is included so the check can run independently.
def calculate_stress(force_n, area_m2):
    if area_m2 <= 0:
        raise ValueError("Area must be positive, in m^2.")
    return force_n / area_m2  # Uniform axial stress in Pa.

# Obtain actual and expected values independently.
actual_pa = calculate_stress(10.0, 2.0)
expected_pa = 5.0
# A false condition raises AssertionError with this diagnostic message.
assert actual_pa == expected_pa, "Known stress case failed"
print("Known-answer check passed")
```

Code 3.9: Slide example: an executable expectation

Example file: `ex37_assert_known.py`

```
Known-answer check passed
```

The comma introduces the assertion's message; it is not an instruction to print two values. The message appears when the claim fails, not every time the statement is executed. The final print is explicit confirmation that execution reached the end of this selected check.

To observe a failing assertion without damaging the working example, run the separate intentional-error file below.

Worked demonstration: a failing assertion.

```python
# Intentional failed expectation to illustrate AssertionError.
actual_pa = 4.0
expected_pa = 5.0
# Unlike a print of False, this stops execution in normal test mode.
assert actual_pa == expected_pa, "Known stress case failed"
# This line is not reached when the assertion is active.
print("This is not a passing test")
```

Code 3.10: Worked demonstration: a failing assertion

Example file: `ex48_assertion_failure.py`

In normal test mode, it raises `AssertionError: Known stress case failed`. The final print is not reached. As discussed in Section 3.15, optimized execution can omit the assertion, so assertions are not the correct sole protection for required input contracts.

3.12. A test function is an ordinary function

Lecture-slide alignment: Week 7 source PDF, p. 67. PDF page numbers include title and divider pages.

A small suite can be grouped using the `def` syntax already learned in Week 4. No new testing framework is required.

Slide example: four checks grouped in a function.

```python
# Required function from the preceding validated-stress slide.
def calculate_stress(force_n, area_m2):
    if area_m2 <= 0:
        raise ValueError("Area must be positive, in m^2.")
    return force_n / area_m2

# A test function is called like any ordinary Python function.
def test_calculate_stress():
    # Hand-checkable positive force, zero force, and signed force.
    assert calculate_stress(10.0, 2.0) == 5.0
    assert calculate_stress(0.0, 2.0) == 0.0
    assert calculate_stress(-10.0, 2.0) == -5.0
    # This additional known case checks a realistic unit scale.
    actual_mpa = calculate_stress(10000.0, 0.0005) / 1e6
    assert abs(actual_mpa - 20.0) <= 1e-9

# Defining the suite does not run it; this call does.
test_calculate_stress()
print("Four selected stress checks passed")
```

Code 3.11: Slide example: four checks grouped in a function

Example file: `ex38_test_function.py`

```
Four selected stress checks passed
```

The four checks cover a hand-calculated value, zero force, negative force under the signed convention, and a realistic-unit benchmark. The last check converts pascals to megapascals before comparing with 20.0, and states its numerical tolerance.

Defining `test_calculate_stress` does not execute the checks. The explicit call runs them. If an active assertion fails, the function stops at that point rather than continuing silently to its success print. The four checks do not yet include every promise: invalid-area rejection and classifier boundaries still need their own tests.

3.13. A rejection can be the successful result of a test

Lecture-slide alignment: Week 7 source PDF, p. 68. PDF page numbers include title and divider pages.

For invalid input, the expected behavior may be an exception. A test passes only if it observes the specified rejection.

Slide example: successful rejection is a passing test.

```python
# Include the component to be tested in this standalone example.
def calculate_stress(force_n, area_m2):
    if area_m2 <= 0:
        raise ValueError("Area must be positive, in m^2.")
    return force_n / area_m2

try:
    # The specified result for zero area is an exception, not a number.
    calculate_stress(10.0, 0.0)
except ValueError:
    print("Passed: zero area was rejected")
else:
    # No exception would violate the promised rejection behavior.
    raise AssertionError("Expected ValueError for zero area")
```

Code 3.12: Slide example: successful rejection is a passing test

Example file: `ex39_expected_error_test.py`

```
Passed: zero area was rejected
```

The `try` invokes the invalid request. A `ValueError` is the expected observation, so that handler reports a pass. If no exception is raised, the `else` raises `AssertionError`: the contract was not enforced. A different exception is not caught and should be investigated rather than counted as the expected success.

This avoids a common testing mistake: catching every exception and declaring the invalid-input test passed. An unrelated `NameError` does not prove that positive-area validation works. Test the *kind* of failure as well as the presence of some failure.

3.14. A tolerance is part of the test specification

Lecture-slide alignment: Week 7 source PDF, p. 69. PDF page numbers include title and divider pages.

Slide example: numerical tolerances with meaning.

```python
# Convert Celsius to kelvin using the stated offset.
actual_k = 25.0 + 273.15
expected_k = 298.15
# This absolute tolerance applies to a temperature difference in K.
absolute_tolerance_k = 1e-9
assert abs(actual_k - expected_k) <= absolute_tolerance_k
# A second check handles a familiar binary-rounding example.
assert abs((0.1 + 0.2) - 0.3) <= 1e-12
print("Two tolerance checks passed")
```

Code 3.13: Slide example: numerical tolerances with meaning

Example file: `ex40_tolerance_assertions.py`

```
Two tolerance checks passed
```

The Celsius-to-kelvin check compares a temperature difference with an absolute tolerance of

10^{-9} K. This identifies the quantity and unit of the criterion. Printing a value with nine decimal places would not implement the same check.

The second assertion uses the familiar dimensionless addition example. Both numerical tolerances are illustrative checks of arithmetic agreement. They do not automatically describe a sensor's resolution, the uncertainty of a measurement, or the error of a physical model. A useful test specification states why its criterion is adequate for its purpose.

3.15. Input validation is not an assertion

Lecture-slide alignment: Week 7 source PDF, p. 70. PDF page numbers include title and divider pages.

The two mechanisms express different responsibilities.

Purpose	Use	Reason
External or caller input	An explicit condition with `raise`	Required contract checks should remain part of ordinary program behavior.
Known-answer testing	`assert` with an expected result	A failed development expectation should be exposed.
Internal invariant	`assert` with a meaningful condition	The program should reveal an unexpected internal state.

Do not rely on assertions alone to reject invalid user input

Python can omit `assert` statements during optimized execution, for example with `python -O`. Use explicit validation for required input checks. Run the test examples here with assertions enabled, not in optimized mode.

The distinction concerns intent as well as execution mode. A user making a typing mistake is not evidence that the program's own internal reasoning is inconsistent. An internal accounting failure, by contrast, can indicate a defect in a loop that should have maintained a known relation.

3.16. An internal invariant checks the program's accounting

Lecture-slide alignment: Week 7 source PDF, p. 71. PDF page numbers include title and divider pages.

The slide counts already-valid temperatures into normal and warning groups. Each reading should enter exactly one of the two groups.

Slide example: an internal counting invariant.

```python
# The slide assumes that these temperature readings are already valid.
readings_c = [19.0, 22.0, 31.0]
# Counters start at zero before any reading is processed.
normal_count = 0
warning_count = 0
for value_c in readings_c:
    # Inclusive operating limits define the two complementary groups.
    if 20.0 <= value_c <= 30.0:
        normal_count += 1
    else:
        warning_count += 1
# Every input should have contributed to exactly one counter.
assert normal_count + warning_count == len(readings_c)
print(normal_count, warning_count)
```

Code 3.14: Slide example: an internal counting invariant

Example file: `ex41_internal_accounting.py`

```
1 2
```

For 19.0 degrees Celsius, the warning count becomes 1. For 22.0, the normal count becomes 1. For 31.0, the warning count becomes 2. The final relation is

$$N_{\text{normal}} + N_{\text{warning}} = N_{\text{readings}} = 3.$$

The assertion checks this internal accounting. It does not prove the temperatures are valid, that the category labels are physically suitable, or even that every comparison uses the correct threshold. A program could classify each reading into the wrong group and still count all three. Combine the invariant with category and boundary tests.

3.17. Complete the mean-reading test suite

Lecture-slide alignment: Week 7 source PDF, p. 72; also p. 25. PDF page numbers include title and divider pages.

Lecture 19 planned five cases. Lecture 21 supplies a function that rejects empty input and asks students to complete its test suite.

Slide activity: mean and non-mutation.

```python
# Contract: a list of finite numerical readings, left unchanged.
def mean_reading(values):
    # The mean is not defined by this contract for an empty dataset.
    if len(values) == 0:
        raise ValueError("At least one reading is required.")
    return sum(values) / len(values)

readings = [2.0, 4.0, 6.0]
# Copy the original state before invoking the function.
before = readings.copy()
# Check both the returned answer and the non-mutation guarantee.
assert mean_reading(readings) == 4.0
assert readings == before
print("Mean and non-mutation checks passed")
```

Code 3.15: Slide activity: mean and non-mutation

Example file: `ex42_mean_slide.py`

```
Mean and non-mutation checks passed
```

The early `raise` prevents division by zero for an empty list and expresses the intended data policy. The remaining input contract still assumes finite numerical readings; the function does not inspect each element's type or finiteness. `sum()` and `len()` read the list without modifying it.

The worked completion below adds the singleton, equal-value, mixed-sign, and expected-empty-input cases. Each valid input is copied before the call so its non-mutation guarantee is checked too.

Worked solution: complete the mean test plan.

```python
# This function enforces nonemptiness; finite numbers remain a precondition.
def mean_reading(values):
    if len(values) == 0:
        raise ValueError("At least one reading is required.")
    return sum(values) / len(values)

# Four valid cases cover a known answer, singleton, equal, and mixed signs.
cases = [([2.0, 4.0, 6.0], 4.0), ([7.0], 7.0),
         ([3.0, 3.0, 3.0], 3.0), ([-2.0, 0.0, 2.0], 0.0)]
for values, expected in cases:
    before = values.copy()  # Preserve evidence before each call.
    assert mean_reading(values) == expected
    assert values == before  # Test the side-effect contract too.
# The fifth case is an expected failure, not a fabricated mean of zero.
try:
    mean_reading([])
except ValueError:
    print("Passed: empty readings were rejected")
else:
    raise AssertionError("Empty readings must raise ValueError")
print("Four valid mean cases and non-mutation checks passed")
```

Code 3.16: Worked solution: complete the mean test plan

Example file: `ex43_mean_complete_tests.py`

```
Passed: empty readings were rejected
Four valid mean cases and non-mutation checks passed
```

An empty mean is not silently assigned zero. Zero is a valid mean for the mixed-sign example and therefore cannot safely double as an unexplained failure indicator. This is the same missing-versus-zero distinction encountered with dictionary records.

3.18. Successful conversion does not prove numerical validity

Lecture-slide alignment: Week 7 source PDF, p. 73. PDF page numbers include title and divider pages.

Slide example: parseable does not mean finite.

```python
# Import a standard-library tool for inspecting numerical values.
import math
for text in ["12.5", "nan", "inf"]:
    # All three strings are accepted by float().
    value = float(text)
    # Only the first represents a finite numerical reading.
    print(text, math.isfinite(value))
```

Code 3.17: Slide example: parseable does not mean finite

Example file: `ex44_finite_conversion.py`

```
12.5 True
nan False
inf False
```

All three fields can be converted by `float()`. The result `nan` represents "not a number," while `inf` represents positive infinity. Neither is a finite measured force or area for this instructional report. `math.isfinite()` checks the stored numerical value rather than its original text.

This separates two validation questions. *Can the field be parsed?* is a question about representation. *Does the parsed value satisfy the input domain?* is a question about meaning. A robust input stage can need both. For the integrated stress example, all inputs must be finite, and area and allowable stress must also be strictly positive.

3.19. Integrated stress report: contract, calculation, and decision

Lecture-slide alignment: Week 7 source PDF, pp. 74–76. PDF page numbers include title and divider pages.

The final example combines specification, validation, computation, reporting, and tests. It accepts finite force in N, positive finite area in mm^2, and a positive allowable stress in MPa. It returns signed stress, but applies the stated rule to its magnitude. Equality at the allowable stress passes.

The independent benchmark is

$$\sigma = \frac{10\,000\,\mathrm{N}}{80\,\mathrm{mm}^2} = 125\,\mathrm{MPa}.$$

The conversion identity $1\,\mathrm{N/mm}^2 = 1\,\mathrm{MPa}$ allows division directly in the supplied units. For negative force, the returned stress is negative, but its magnitude is compared with the positive allowable value.

Completed teaching implementation of a printed excerpt

The PDF displays the range checks, calculation, finite-result check, decision, and return, but not the preceding type/finiteness checks. The implementation below adds a named function, a documented numeric-type policy, the omitted checks, and a caller. It is a completion based on the printed contract, not a recovered original source file. The displayed calculation and decision behavior are retained.

Completed teaching implementation: contract and validation (part 1 of 2).

```python
# Completed implementation: the slide shows the calculation body only.
import math

def stress_report(force_n, area_mm2, allowable_mpa):
    """Return (signed stress in MPa, stated-rule status).

    Use ordinary built-in int/float inputs, excluding bool.
    All inputs and the calculated stress must be finite.
    Area and allowable stress must be strictly positive.
    The rule compares stress magnitude; equality is accepted.
    This instructional criterion does not certify component safety.
    """
    # These explicit checks complete the prefix not printed in the PDF.
    for name, value in [("force_n", force_n),
                        ("area_mm2", area_mm2),
                        ("allowable_mpa", allowable_mpa)]:
        # bool is deliberately rejected although it behaves numerically.
        if isinstance(value, bool) or not isinstance(value, (int, float)):
            raise TypeError(f"{name} must be an int or float, not bool.")
        # Very large Python integers can exceed floating-point range.
        try:
            finite = math.isfinite(value)
        except OverflowError:
            raise ValueError(f"{name} exceeds the supported numeric range.")
        if not finite:
            raise ValueError(f"{name} must be finite.")
```

Code 3.18: Completed stress report: contract and validation

Same file continued: function body and caller (part 2 of 2).

```
    # The following calculation/decision follows the slide excerpt.
    if area_mm2 <= 0 or allowable_mpa <= 0:
        raise ValueError("Area and allowable stress must be positive.")
    # N/mm^2 equals MPa; retain the sign in the returned stress.
    stress_mpa = force_n / area_mm2
    # Finite inputs can still produce an unrepresentable ratio.
    if not math.isfinite(stress_mpa):
        raise ValueError("Calculated stress is not finite.")
    # Use magnitude for this illustrative signed-stress criterion.
    if abs(stress_mpa) <= allowable_mpa:
        status = "PASS stated rule"
    else:
        status = "FAIL stated rule"
    return stress_mpa, status

# One caller handles only the specified input/value failures.
try:
    stress_mpa, status = stress_report(10000.0, 80.0, 125.0)
except (TypeError, ValueError) as error:
    print(f"Cannot produce a stress report: {error}")
else:
    # Never print this successful report after a failed calculation.
    print(f"Stress = {stress_mpa:.2f} MPa")
    print(status)
```

Code 3.19: Completed stress report: calculation and reporting, continued

Example file: `ex45_integrated_stress.py`

```
Stress = 125.00 MPa
PASS stated rule
```

3.19.1. Read the validation prefix in order

The loop pairs each parameter name with its value so the message identifies the failing quantity. Ordinary built-in integers and floats are accepted; Booleans are explicitly rejected. This check is needed because Python Boolean objects have numerical behavior, but `True` is not an appropriate measured force in the specified interface.

Finiteness is checked before the physical ranges. The short `try/except OverflowError` addresses extremely large Python integers that the floating-point check cannot represent. That is an added range safeguard in this completed implementation, not a line shown in the slide excerpt.

After the prefix, nonpositive area and nonpositive allowable stress are rejected. Division then computes stress in MPa. Even with finite inputs, an extreme ratio can exceed floating-point range; the finite-result check prevents infinity from becoming a reported stress.

3.19.2. Read the decision and caller separately

The comparison uses `abs(stress_mpa)`, while the returned stress retains its sign. The two status strings preserve the slide's wording: `"PASS stated rule"` and `"FAIL stated rule"`. A returned FAIL is a *valid calculation that exceeds the supplied criterion*, not a program exception.

The caller catches the documented input/type failures and prints an explanation. The `else`

prints stress and status only after the function has returned normally. A rejected input therefore cannot be followed by a fabricated stress or a PASS label.

Interpret the limited claim

This is a simplified uniform axial-stress calculation with an illustrative magnitude criterion. It does not certify a real component, establish different tensile/compressive allowable stresses, or validate the supplied material and load data. A mathematically valid PASS still depends on the appropriateness of the model and criterion.

3.19.3. The integrated test matrix

Input change from the baseline	Expected behavior	Purpose
10 000 N; 80 mm^2; limit 125 MPa	125 MPa; PASS	Known answer and equality.
Force = 10 008 N	125.1 MPa; FAIL	Just above the criterion.
Force = 0 N	0 MPa; PASS	Zero-load boundary.
Area = 0 or −1 mm^2	`ValueError`	Invalid geometry.
NaN or infinity	Rejection	Nonfinite numeric data.
Boolean input	Rejection	Wrong kind of measurement.

These are the source matrix cases. For the completed implementation, nonfinite values raise `ValueError` and Boolean values raise `TypeError`. Additional worked checks include signed force, nonpositive allowable stress, and a nonfinite calculated ratio. Those make the chosen implementation's contract explicit without adding new assessment requirements.

3.20. Lecture 21 exit ticket and worked answers

Lecture-slide alignment: Week 7 source PDF, p. 77. PDF page numbers include title and divider pages.

Why reject nonpositive area rather than return zero? A zero return could be mistaken for a valid zero-stress result. Rejection states that the requested calculation has no justified result under the input contract.

When does the optional else run? After the protected `try` suite completes normally. It is skipped when a failure transfers control to a handler. The simple examples here have no early `return`, `break`, or `continue` from the protected block.

What is appropriate for user input? Explicit validation with a meaningful exception. Assertions are useful development checks but can be omitted in optimized execution.

Give one successful-value and one successful-error test.

The known call `calculate_stress(10.0, 2.0)` should return 5.0 Pa. The call with area 0.0 should raise the documented `ValueError`; absence of that exception is a failed test. Sections 3.11 and 3.13 show the fully commented implementations.

Lecture 21 in one paragraph

Input contracts define when a calculation is meaningful. Explicit validation rejects invalid requests. Narrow handlers communicate or recover from expected failures, while unexpected defects remain visible. Assertions make selected development expectations executable. Test values, units, boundaries, data preservation, and expected rejection together before interpreting the final report.

4 Optional Extensions and Week 7 Synthesis

Lecture-slide alignment: Week 7 source PDF, pp. 78–87. PDF page numbers include title and divider pages.

The closing slide pages collect optional tools and summarize the week's habits. They do not replace the core material. In particular, a testing framework or a graphical debugger is not required to understand why a particular expectation fails.

4.1. Optional: finally is for cleanup

Lecture-slide alignment: Week 7 source PDF, p. 78. PDF page numbers include title and divider pages.

A `finally` clause specifies an action to perform on leaving the `try` statement within ordinary Python control flow. In contrast, `else` is reserved for successful normal completion of the protected suite.

Slide optional example: cleanup is not a success label.

```
# Compare the failing field with "12.5" in a separate run.
text = "bad"
try:
    value = float(text)  # May raise the expected ValueError.
except ValueError:
    print("Cannot convert")
else:
    print("Converted", value)  # Only the successful route reports value.
finally:
    # Within ordinary control flow, this runs on either route.
    print("Check finished")
```

Code 4.1: Slide optional example: cleanup is not a success label

Example file: `ex46_finally.py`

```
Cannot convert
Check finished
```

With `"bad"`, the handler prints `Cannot convert` and the final clause prints `Check finished`. With `"12.5"`, the `else` prints the converted value and the final clause still prints `Check finished`. That final message records completion of the cleanup/checking action, not success of the conversion.

In larger programs, the same idea can be used to release a resource after an attempted operation. The source confines this topic to a control-flow preview. It does not require file handling or resource-management mechanisms in this week's assessed work. "Finally runs" describes ordinary language execution, not a guarantee against a forcibly terminated process or a machine losing power.

4.2. Optional: combine absolute and relative tolerance

Lecture-slide alignment: Week 7 source PDF, p. 79. PDF page numbers include title and divider pages.

An absolute criterion compares a difference with a fixed scale. A relative criterion adjusts to the magnitudes of the values. The slide introduces the combined rule used by `math.isclose` for finite values:

$$|a - b| \leq \max(\varepsilon_{\text{rel}} \max(|a|, |b|),\ \varepsilon_{\text{abs}}) .$$

The relative tolerance is dimensionless. The absolute tolerance has the unit of the compared quantity and is particularly useful near zero.

Slide optional example: relative and absolute tolerance.

```
# Standard-library comparison tool; no third-party package is needed.
import math
actual = 1000000.001
expected = 1000000.0
# Relative tolerance scales with the magnitude of the inputs.
print(math.isclose(actual, expected, rel_tol=1e-8, abs_tol=1e-9))
# Near zero, the absolute tolerance provides the useful comparison scale.
print(math.isclose(1e-12, 0.0, rel_tol=1e-8, abs_tol=1e-9))
```

Code 4.2: Slide optional example: relative and absolute tolerance

Example file: `ex47_isclose.py`

```
True
True
```

In the first comparison, a difference of roughly 0.001 is small relative to values near one million. The relative allowance is approximately $10^{-8} \times 10^{6} = 0.01$, so the check passes. In the second comparison, the absolute floor 10^{-9} accepts a difference of 10^{-12} from zero.

These illustrative numbers are not universal accuracy requirements. State which quantity is being compared, why its scale matters, and how the chosen tolerance relates to the purpose of the test. Do not use a forgiving tolerance to conceal a wrong unit conversion.

4.3. Optional: inspect state with a graphical debugger

Lecture-slide alignment: Week 7 source PDF, p. 80. PDF page numbers include title and divider pages.

A debugger makes execution observable without inserting a print for every value. Set a breakpoint just before a suspect calculation, inspect the parameters and local variables, and compare them with the expected state. *Step into* follows a function call into its body; *step over* runs a call while keeping the current level visible so that its result can be inspected.

For the bar example, a useful breakpoint is immediately after area conversion. The predicted value is 0.0005 m^2. If the debugger displays 0.5, the error has already occurred; there is little value in first examining the final classification. Stop at the earliest divergence, repair its cause, and rerun the retained failing test.

The slides deliberately describe a tool-independent method rather than a particular editor's buttons or keyboard shortcuts. A trace table, a labelled printout, and a debugger all investigate the same program state.

4.4. Worked extension: add finite-data checking to retry

Lecture-slide alignment: Week 7 source PDF, pp. 64 and 73, combined as a reading-note extension. PDF page numbers include title and divider pages.

This is a new combination of two source examples, not the original retry listing. It keeps the cancellation and conversion-handling structure but adds `math.isfinite()` before accepting a positive area.

Worked extension: add finiteness to the retry example.

```
# This extension combines two separate slide ideas explicitly.
import math
while True:
    text = input("Area in m^2 (or q): ").strip()
    if text.lower() == "q":
        print("Cancelled")
        break
    try:
        area_m2 = float(text)  # Check the number's textual representation.
    except ValueError:
        print("Enter a number.")
        continue
    # Reject nonfinite values before applying a positive-area criterion.
    if not math.isfinite(area_m2) or area_m2 <= 0:
        print("Area must be finite and positive.")
        continue
    print("Accepted:", area_m2, "m^2")
    break
```

Code 4.3: Worked extension: add finiteness to the retry example

Example file: `ex49_finite_retry.py`

```
Area in m^2 (or q): nan
Area must be finite and positive.
Area in m^2 (or q): inf
Area must be finite and positive.
Area in m^2 (or q): 0.002
Accepted: 0.002 m^2
```

The displayed sample rejects `nan` and `inf` before accepting 0.002 m^2. This closes the specific preview limitation identified in the source retry example. It still does not determine whether 0.002 m^2 is a physically appropriate area for a particular real object; that belongs to the input specification and engineering context.

4.5. Quick reference: plan tests, then diagnose failures

Lecture-slide alignment: Week 7 source PDF, p. 81. PDF page numbers include title and divider pages.

Need	Use	Ask
Expected behavior	Specification and an independent example.	What should happen, in which units?
Input selection	Typical, boundary, and invalid cases.	Which meaningful behavior classes are missing?
Code paths	Black-box plus glass-box tests.	Which branches, exits, or returns remain untested?
Fault localization	Traceback, values, types, and units.	Where is the first wrong state?
Repair evidence	Rerun and retain regression cases.	Did this change alter another behavior?

A useful test record contains a case name, input, expected value or exception, actual observation, decision criterion, and result. A debugging record adds the suspected cause, one change, and the evidence observed after the change. These records make it possible to learn from a defect instead of merely forgetting it after one successful run.

4.6. Quick reference: exceptions and assertions

Lecture-slide alignment: Week 7 source PDF, p. 82. PDF page numbers include title and divider pages.

Construct	Role in this week's programs
`raise`	Reject an invalid value or report a failure explicitly.
`try` / `except`	Protect a chosen operation and handle a matching failure.
`as error`	Access the caught exception within its handler.
`else`	Follow up after the protected suite completes normally.
`assert`	Check a development/test expectation or internal invariant.
`finally`	Carry out optional cleanup when leaving the statement in ordinary control flow.

Key idea

Keep failure visible. Required input checks should use explicit validation. Do not replace a failed engineering calculation with a plausible-looking numerical answer. Catch an exception because you have a defined response to it, not because you dislike seeing a traceback.

4.7. Concept map: from a promise to a reproducible report

Lecture-slide alignment: Week 7 source PDF, p. 83. PDF page numbers include title and divider pages.

The slide concept map links a specification to planned tests and observed evidence. A failed case motivates debugging. Validation and exceptions make input and failure behavior explicit; assertions and regression tests preserve expectations. The final objective is a reproducible report whose claim matches its evidence.

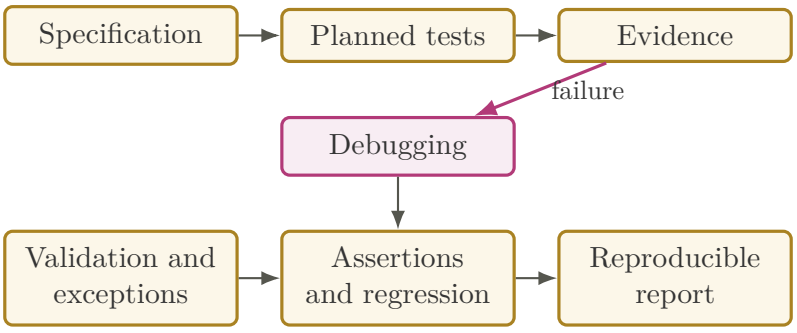


Figure 4.1.: Reading-note rendering of the Week 7 concept map. The components support one workflow, not isolated syntax topics.

For example, the integrated stress report begins with units and valid ranges, checks a 125 MPa equality case, rejects invalid area, exposes wrong units in staged debugging, and retains the repaired cases. None of these tasks substitutes for the others.

4.8. Scope, assessment, and preparation for the next topic

Lecture-slide alignment: Week 7 source PDF, pp. 84 and 86. PDF page numbers include title and divider pages.

Week 7 covers classes of test, testing approaches, debugging steps, error messages, exceptions, assertions, and documented engineering evidence. Later units develop classes, data attributes, methods, inheritance, encapsulation, and formal algorithmic efficiency. These notes do not pull those later topics into the core Week 7 requirements.

Follow the separate Moodle assessment announcements. The examples here do not create new assignment requirements or change examination coverage. The reading-note exercises are for learning and self-checking unless the instructor separately specifies otherwise.

Before the next topic, run the working examples from a clean session, attempt a faulty starter before opening its repair, explain the expectation behind every test and the purpose of every caught exception, and retain the inputs that exposed a defect. A framework cannot compensate for an unclear contract or an incorrect expected answer.

4.9. The closing engineering principle

Lecture-slide alignment: Week 7 source PDF, p. 87. PDF page numbers include title and divider pages.

A trustworthy result comes with evidence

State the contract. Challenge the boundaries. Explain the failure. Repair one cause. Keep the test.

The goal is not a program that always prints a number. The goal is a program that produces a justified result when its requirements hold, reports honestly when they do not, and provides enough evidence for another person to understand the difference.

5 Cumulative Practice and Worked Solutions

The questions below are additional reading-note practice built from the slide examples. They are not new assessed tasks. Work through the questions without running code first, then use the referenced commented examples to test your reasoning.

5.1. Twelve self-check questions

Q1. A function promises to accept FoS ≥ 2.0 but uses `fos > 2.0`. Give three tests that expose the intended lower boundary. Explain why two ordinary examples are insufficient.

Q2. A finite-temperature classifier uses inclusive limits 20 and 30 degrees Celsius. What is the difference between black-box and glass-box test design? Can three tests execute all branches while missing an equality defect?

Q3. Trace the faulty `total_load` function in Code 1.5 for an empty list, a singleton 5.0, and the values 2.0, 3.0, 4.0. State the expected and observed totals.

Q4. A test compares the result of 0.1 + 0.2 with 0.3 using exact equality. Explain the observation and replace the test criterion with the slide's absolute-tolerance rule. Does that tolerance describe sensor accuracy?

Q5. Another stress implementation returns $1000F/A$. Which of the doubling-force, doubling-area, zero-force, and absolute-known-answer tests can expose the constant factor? Explain.

Q6. A traceback ends at `return force_n / area_m2` with division by zero. What should you inspect in the caller? Why is replacing the denominator with a tiny arbitrary number not a sound repair?

Q7. For the faulty bar example, derive the correct area conversion and expected stress/FoS. Why does fixing the name mismatch alone leave an incorrect program?

Q8. A calibration returns the expected processed values, but changes the original list. Design a test of the violated promise. Why is `before = raw_c` inadequate evidence?

Q9. Predict the printed letters in the conversion-trace example for `"bad"` and `"12.5"`. Does a handled exception resume execution at the failed conversion?

Q10. Why is returning 0.0 after every caught stress-calculation exception misleading? Contrast this with returning a valid `"FAIL stated rule"` status from the integrated report.

Q11. Design a test that passes when zero area raises `ValueError` and fails when no exception occurs. Why must an unrelated `NameError` not count as a passing rejection test?

Q12. For the completed integrated report, predict the result for forces 10 000, 10 008, 0, and −10 000 N with area 80 mm^2 and limit 125 MPa. Then predict the behavior for zero area,

NaN force, and Boolean force. Identify which exception-type choices belong to the completed implementation rather than the source matrix alone.

5.2. Worked solutions

Q1. The boundary belongs to the contract

Use 1.999, 2.0, and 2.001. The expected labels are not acceptable, acceptable, and acceptable. The faulty implementation rejects the equality case. Ordinary values such as 1.5 and 2.5 do not distinguish strict from inclusive comparison. The repaired commented function is Code 1.2; retain 2.0 as a regression case.

Q2. Requirements and code paths are complementary views

Black-box tests come from the specified valid domain, categories, and boundary rules without reading the body. Glass-box tests use the visible conditions and returns to choose paths. Values 15, 25, and 35 can enter all three categories without testing either equality. Include 20 and 30, plus neighboring values, to test the contract's inclusiveness. Section 1.5 reproduces the complete six-case matrix.

Q3. The empty case alone is misleading

The actual totals are 0.0, 0.0, and 5.0 N. The expected totals are 0.0, 5.0, and 9.0 N. The loop stops before the last index because both `len(loads_n) - 1` and the excluded endpoint of `range` reduce the visited positions. Direct iteration over values repairs the omission; see Code 1.6.

Q4. Specify a numerical acceptance rule

The stored sum is normally displayed as `0.30000000000000004`, so exact equality with the stored 0.3 is false. Check $|x - 0.3| \leq 10^{-12}$ for this arithmetic exercise. The tolerance is a criterion for numerical agreement, not a statement that a temperature or stress sensor measures to 10^{-12} in any unit. Code 1.8 prints both comparisons.

Q5. Scaling alone cannot detect a constant multiplier

The wrong function still doubles when force doubles, halves when area doubles, and returns zero at zero force. Those three checks pass. The independent case $F = 10$ N, $A = 2$ m^2 should return 5 Pa, whereas the wrong function returns 5000 Pa; that known-answer test exposes the defect. Different test classes challenge different misconceptions.

Q6. Follow the input back to its origin

Inspect the actual value of `area_m2`, the argument supplied at the call site, and the earlier parsing or conversion that produced it. The division reports the symptom; the bad denominator may originate elsewhere. Replacing zero with an arbitrary tiny value invents geometry and therefore a stress. Explicitly reject the invalid area under the stated contract instead.

Q7. Both geometric and stress units must be aligned

Because 1 mm $= 10^{-3}$ m, 1 mm$^2 = 10^{-6}$ m^2. Thus 500 mm$^2 = 0.0005$ m^2, stress is 20×10^6 Pa $= 20$ MPa, and FoS is $250/20 = 12.5$. Repairing `Force`/`force` only allows later arithmetic to execute. It does not change either wrong conversion. Section 2.8 shows each stage and its separate evidence.

Q8. Inspect the state before and after the call

For the flat list `[22.0, 23.0]`, save `before = raw_c.copy()`, apply the offset correction, and compare `raw_c` with `before`. Also check that the returned values are `[21.5, 22.5]`. Assigning `before = raw_c` would merely create an alias, so both names could change together and hide the mutation. Code 2.14 contains the commented test.

Q9. A handler changes the execution route

For `"bad"`, output is A, C, D. For `"12.5"`, output is A, B, D. A conversion failure skips the remaining statements in `try`, runs the matching handler, and continues after the handled block. It does not automatically try the failed conversion again. A retry requires an enclosing loop or another explicit call.

Q10. Invalid computation and an exceeded criterion differ

Returning zero after an exception disguises the absence of a valid calculation. By contrast, `"FAIL stated rule"` from the integrated function accompanies a successfully calculated finite stress whose magnitude exceeds the supplied limit. A valid failing design check should still report the computed stress and criterion. Invalid input should produce rejection, not a fabricated result.

Q11. Check the promised exception, not any exception

Protect the invalid request with `try`, catch `ValueError`, and use `else` to raise `AssertionError` if no failure occurs. Leave a different exception uncaught so it remains visible. A `NameError` shows that a name is missing, not that area validation works. Code 3.12 is the complete commented pattern.

Q12. Signed output, magnitude criterion, explicit rejection

Case	Completed implementation's result
10 000 N	125 MPa; `PASS stated rule`.
10 008 N	125.1 MPa; `FAIL stated rule`.
0 N	0 MPa; `PASS stated rule`.
−10 000 N	−125 MPa; `PASS stated rule`, because equality holds for the magnitude.
Zero area	`ValueError`; no stress is reported.
NaN force	`ValueError`; the value is not finite.
Boolean force	`TypeError`; a Boolean is excluded from the chosen numeric-input policy.

The source matrix explicitly requires `ValueError` for nonpositive area, and rejection for NaN, infinity, and Boolean inputs. It does not give every exception class for those final categories. The exact TypeError/ValueError allocation is documented in the completed teaching implementation.

5.3. A readiness checklist

Before moving on, explain a test's expected result without calling the function; distinguish a boundary case from a code path; read a traceback bottom-up; locate a unit error using an intermediate value; test that a list was not mutated; trace the roles of `try`, `except`, and `else`; and distinguish an input check from an assertion. Finally, run your examples from a clean session and verify that a colleague could reproduce your evidence.

Key idea

Being able to explain *why a test should fail* is as important as being able to produce a passing program. That explanation connects the requirement, the code, and the evidence.

A Glossary and Technical Clarifications

A.1. Week 7 glossary

Specification
The promised input domain, units, behavior, returned result, and failure response against which implementation is judged.

Contract
The agreement between caller and function, including assumptions and guarantees. Some assumptions may be the caller's responsibility rather than checks enforced by code.

Test case
A chosen input and an independently specified expected result or failure, together with an acceptance criterion.

Test suite
A collection of selected checks retained for repeated execution. Passing it is evidence about those checks, not a universal proof.

Known-answer test
A comparison with a result established independently, often by hand calculation or a simple benchmark.

Boundary test
A case at or near a change in validity or category, including exact equality where relevant.

Invalid-input test
A case whose expected outcome is the documented rejection of input outside the contract.

Sanity check
A check of magnitude, sign, or a physically expected trend; useful but not a replacement for an absolute benchmark.

Black-box testing
Selecting cases from the specification without relying on the implementation's internal structure.

Glass-box testing
Selecting cases after inspecting branches, conditions, loop exits, and recursive return paths.

Coverage
Evidence that chosen code structures have been exercised. Execution of a path does not establish that its calculation or comparison is correct.

Unit test
A test focused on one small program component or function.

Integration test
A test of how components connect, including unit, meaning, and data-interface agreement.

Regression test
A retained case used to detect whether a previously repaired behavior has broken again after later changes.

Test driver
Code that supplies test inputs, calls a component, compares observations with expectations,

and reports outcomes.

Debugging
A controlled investigation that reproduces a symptom, locates its cause, repairs it, and retests the result.

Minimal reproducible example
The smallest useful program/data combination that still demonstrates the same failure.

Traceback
The call trail reported for a failure, identifying the exception and the source locations involved.

Symptom
An observed failure, such as an exception, unexpected output, or nonterminating run. Its cause may occur earlier.

Program state
The values and objects relevant at a particular point in execution, including parameters, local variables, and referenced data.

Side effect
A change observable beyond a function's returned value, such as mutation of a caller-supplied list.

Clean run
Execution from a restarted session with all required definitions and inputs supplied by the saved program.

Exception
A signal that interrupts normal execution and can transfer control to a matching handler or propagate outward.

Handler
The code in an `except` clause that responds to a matching exception.

Validation
Explicit checks that input satisfies the contract before it is used in a calculation.

Assertion
An executable test/development claim that raises `AssertionError` if false when assertions are active.

Invariant
A relation expected to hold at a specified point during execution, such as total accounted readings equaling the number processed.

Absolute tolerance
An accepted numerical difference expressed on the same scale and in the same unit as the compared quantity.

Relative tolerance
A dimensionless accepted difference scaled by the magnitudes of the compared values.

Finiteness
The property of being neither NaN nor positive/negative infinity in the numerical checks used here.

A.2. Clarifications and source boundaries

PDF layout is not Python syntax. Several slide code lines wrap at the panel boundary. The source sometimes displays a comment continuation or a function call across visual lines. The annotated examples restore valid logical lines without changing intended operations. Deliberate syntax and logic defects remain deliberate and labelled.

Assumptions versus validation. The early stress and temperature functions and the mean-reading function retain their declared finite-numerical-input assumptions. Their code does not enforce every assumption. The initial retry loop similarly retains the source's finite-input limitation. The completed stress report and finite-retry extension are labelled additions, not silent replacements.

A complete starter is not present in the PDF. The activity on page 49 specifies three defect categories and a numerical benchmark. The reconstructed starter and its solution are supplied to make that activity usable as a reading exercise. Their implementation is not claimed to be the original file named by the slide.

The integrated report is an excerpt in the PDF. The initial type/finiteness checks and its caller are supplied as a completed teaching implementation. Booleans are explicitly rejected. The additional handling of extremely large Python integers is a representational safeguard. The report is not a general-purpose numerical library: successful finite arithmetic alone does not establish all relative-error or underflow requirements for arbitrary extreme inputs.

Zero and signed force. The simple stress functions return signed F/A and allow zero force. The integrated criterion explicitly uses stress magnitude. This does not assert that every physical component has identical tensile and compressive limits.

Copying and saved-state tests. A shallow copy is sufficient for the source's flat lists of immutable numerical readings. It is not a universal guarantee that nested mutable data have been duplicated independently.

Source expected errors. The exact text, line numbers, caret markers, and path prefixes in tracebacks may vary with Python version and file layout. Tests in the accompanying package check the intended exception categories; the source's displayed traceback remains an explicitly abbreviated example.

Success clauses and cleanup. The core `else` examples have ordinary fall-through control flow. The general rule is normal completion of the `try` suite, not merely the absence of a caught exception. `finally` describes cleanup during ordinary execution; it is not evidence of a valid numerical result or protection against external termination.

Assertions and optimization. Required validation uses explicit conditions and exceptions. The verification script checks the distinction separately, but the teaching examples containing `assert` should be run without `-O`.

Teaching criteria are limited claims. FoS = 2.0, the 20–30 degree temperature interval, pressure categories, and supplied material properties are instructional values. These notes do not convert them into certified safety criteria or alter the authoritative Moodle assessment instructions.

B Source Basis and Slide-by-Slide Coverage

B.1. Source basis and suggested reading

The requested basis for this document is the attached presentation by Yanwei Wang, *Programming for Engineers: Week 7 — Testing, Debugging, Exceptions, and Assertions*, Fall 2026, 87 PDF pages (uploaded as `week7_full(1).pdf`). Its three lectures, activities, examples, optional extensions, and source framing are preserved. Page-specific alignment lines and the map below identify that basis throughout the notes.

The presentation itself recommends the following course readings: the *ENG 101 Course Specification Report*, Session 7, supporting CLOs 1, 3, and 5; Y. Wang's *Programming for Engineers with Python*, supplied working draft labelled `v20260815`, Chapter 8, especially Sections 8.2–8.14 and 8.18; and J. V. Guttag's *Introduction to Computation and Programming Using Python*, second edition, Chapters 6–7. These are retained as the slide deck's suggested readings, not used as a substitute syllabus.

B.2. Language-reference checks

The official Python references named by the presentation were also consulted on 13 September 2026 to check the narrow language-behavior clarifications: exception propagation and the successful/cleanup clauses [P1], assertion behavior in optimized execution [P2], and the finiteness and tolerance tools [P3]. Engineering thresholds, datasets, and activity framing come from the supplied slides unless explicitly labelled as a worked completion.

B.3. Coverage of every source PDF page

The left column uses PDF page numbers, including cover and divider pages; it does not use the Beamer footer counters. "Note location" is a numbered section or chapter in this reading document. Front-matter pages are identified by their page number. Related source pages may be developed together to avoid artificial repetition.

PDF page	Source topic	Note location
1	Title and course identity	Page 2
2	Week 7 at a glance and learning progression	Page 3
3	Lecture 19 divider	1
4	Lecture 19 learning goals	1
5	Two passing tests are not enough	1.1
6	Correct with respect to what?	1.2
7	Testing and debugging have different jobs	1.2
8	A test as an engineering experiment	1.3
9	A hand-checkable stress case	1.3
10	Choose inputs for different reasons	1.4

PDF page	Source topic	Note location
11	Scope and repeatability: unit, integration, regression	1.4
12	Black-box testing: the contract	1.5
13	Partition the temperature input space	1.5
14	Temperature boundary matrix	1.5
15	Glass-box testing: inspect paths	1.6
16	Combine contract and path tests	1.6
17	Zero, one, and several repetitions	1.7
18	Faulty total-load activity	1.7
19	Recursive factorial return-path tests	1.8
20	Absolute tolerance and floating-point results	1.9
21	Engineering scaling and sign tests	1.10
22	Test collections, dictionaries, and preserved data	1.11
23	Integration and unit interfaces	1.12
24	A repeatable temperature test driver	1.13
25	Five-case mean-reading test plan	1.14
26	Retain repaired defects as regression cases	1.15
27	Lecture 19 pressure-classifier exit ticket	1.16
28	Lecture 20 divider	2
29	Lecture 20 learning goals	2
30	Bar script with name and unit defects	2.1
31	Classes of error and first inspection actions	2.2
32	Missing-colon syntax error	2.3
33	A runtime failure and its call trail	2.4
34	Reading a traceback bottom-up	2.4
35	Common exception categories	2.5
36	Controlled debugging workflow	2.6
37	Reproduce and reduce a failing example	2.6
38	Inspect values, types, and invisible characters	2.7
39	Bar repair 1: fix the name	2.8
40	Bar repair 2: derive area conversion	2.8
41	Bar repair 3: align strength and stress units	2.8
42	A debugging log connects changes to evidence	2.8
43	Accumulator reset and intermediate-state tracing	2.9
44	A missing return and a downstream TypeError	2.10
45	List-aliasing side-effect defect	2.11
46	Dictionary lookup and material-name mismatch	2.12
47	Loops that do not finish and stopping evidence	2.13
48	Clean runs and hidden notebook state	2.13
49	Tiny-program debugging activity (starter not displayed)	2.14
50	Evidence-based help and responsible AI assistance	2.15
51	Lecture 20 exit ticket	2.16
52	Lecture 21 divider	3
53	Lecture 21 learning goals	3
54	Possible responses to zero area	3.1
55	Normal and exceptional paths	3.1
56	Raise ValueError before invalid arithmetic	3.2
57	Choose a meaningful exception category	3.3

PDF page	Source topic	Note location
58	Handle an expected conversion failure	3.4
59	Two routes through try/except	3.4
60	Printed-letter control-flow activity	3.5
61	Conversion and division handlers	3.6
62	Use else for successful follow-up	3.7
63	The caller handles a function error	3.8
64	Retry, reject, accept, or cancel	3.9
65	Broad handler that fabricates zero	3.10
66	Assertions make expectations executable	3.11
67	An ordinary test function	3.12
68	Test that invalid input is rejected	3.13
69	A tolerance is part of the test contract	3.14
70	Validation versus assertions and optimized execution	3.15
71	Counting invariant	3.16
72	Complete the mean-reading suite	3.17
73	Conversion versus finiteness	3.18
74	Integrated stress report: contract and benchmark	3.19
75	Integrated stress report: calculation excerpt	3.19
76	Integrated success/rejection matrix	3.19
77	Lecture 21 exit ticket	3.20
78	Optional finally and cleanup	4.1
79	Optional absolute and relative tolerance	4.2
80	Optional debugger-based inspection	4.3
81	Testing/debugging quick reference	4.5
82	Exception/assertion quick reference	4.6
83	Week 7 concept map	4.7
84	Scope, assessment authority, and later topics	4.8
85	Source basis and suggested reading	B.1
86	Language references and preparation	B.2
87	A trustworthy result comes with evidence	4.9

B.4. Using the accompanying code files

The source package contains 49 annotated examples, with a manifest that records their source pages and whether each is a slide example, worked repair, completed implementation, interactive exercise, or intentional defect. Seven files raise an expected unhandled exception. Seven further runnable files deliberately demonstrate incorrect results or misleading error handling. Treat neither category as a working engineering solution.

The remaining examples include two interactive retry exercises; their stored transcripts use specified sample inputs. The self-contained combined demonstration script runs 33 noninteractive, non-defective examples in separate function scopes. This prevents one example's global state from making another appear to work. The independent example files remain the simplest way to study one concept at a time.

The verification script checks the observed outputs, expected exception categories, selected

independent answers, source limitations, repaired boundaries, non-mutation promises, finite-data rejection, and the distinction between assertion removal and explicit validation. Its checks do not constitute proof of every possible input or of real engineering safety.

Recommended final reading task

Select one defect from each lecture. For each, write the promised behavior, the smallest revealing input, the actual observation, the first wrong state or failed condition, the repair, and the retained regression test. This small record connects the whole week.

Bibliography

[S] Yanwei Wang. *Programming for Engineers: Week 7 — Testing, Debugging, Exceptions, and Assertions*. Nazarbayev University, School of Engineering, Fall 2026. Supplied lecture presentation, 87 PDF pages.

[P1] Python Software Foundation. *The Python Tutorial: Errors and Exceptions*, Sections 8.1–8.4 and 8.7. https://docs.python.org/3/tutorial/errors.html.

[P2] Python Software Foundation. *The Python Language Reference: The assert statement*, Section 7.3. https://docs.python.org/3/reference/simple_stmts.html#the-assert-statement.

[P3] Python Software Foundation. *The Python Standard Library: math*, entries for `isfinite` and `isclose`. https://docs.python.org/3/library/math.html.